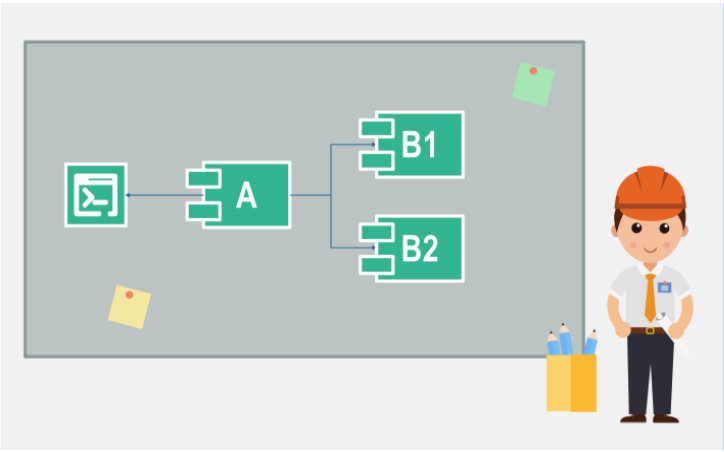
Team 4

Software Architecture Document (SAD)

Terrapin Events



CLIN:	001
Version #:	1.0
Version Date:	11/08/2025
Team Name:	Team 4
Lead Author:	Teja Tatimatla
Contributing Author(s):	Abhigna Kandala, Bhavna Kumari, Heena Khan, Vineet Agarwal

Copyright © 2025 Tony D Barber. All rights reserved.

Table of Contents

Table of Contents **1**
Table of Figures **4**
Table of Tables **5**
1 Introduction **6**
1.1 Document Purpose 6
1.2 Relationship to Other Artifacts (SRS, SADD) 6
1.3 Project Scope 7
1.4 Intended Audience 8
Document Usage Mapping (Extra Credit Attempted) 8
2 Architecture Overview **10**
2.1 Architecture Context and Rationale 10
2.2 Stakeholders and Architectural Concerns 10
2.3 Quality Attribute Goals 11
Performance 11
Scalability 11
Security 12
Availability & Reliability 12
Maintainability 12
Usability & Accessibility 12
Compliance 12
3 Architectural Design **14**
3.1 Architectural Styles and Patterns 14
Design Alternatives vs. Chosen Style: (Extra Credit Attempted) 15
Hybrid Pattern Note (Broker/Event-Driven inside Layered) (Extra Credit Attempted) 15
3.2 Architectural Views 16
Quality Attribute Scenarios (Extra Credit Attempted) 17
Utility Tree Reference (Traceability) (Extra Credit Attempted) 18
3.2.1 Logical View 19
Interface Table (Extra Credit Attempted): 19
3.2.2 Process View 21
Sequence Diagram: (Extra Credit Attempted) 21
Concurrency Note (Runtime & Isolation Model) (Extra Credit Attempted) 22
Performance Optimization Note (What keeps p95 ≤ 2s) (Extra Credit Attempted) 23
3.2.3 Development View 23
3.2.4 Deployment View 24
3.3 System Interfaces and Dependencies 27
3.3.1 API Endpoint Overview (Extra Credit Attempted): 27
3.3.2 System Interfaces Table: 27
System Interfaces 27

Copyright © 2025 Tony D Barber. All rights reserved.

3.4 Architectural Constraints....28
Architectural Constraints....28
3.4.1 Constraint Trace Mapping (Extra Credit Attempted)....29
Layered Traceability Flow (Extra Credit Attempted)....29
4 Architecture Models and Diagrams....30
4.1 Context Diagram....30
4.1.1 Description:....30
4.1.2 Work Context Entity, Roles and Interaction Segregation....30
4.2 Component Diagram....32
4.2.1 Component Interface Reference Table....32
4.3 Deployment Diagram....36
4.3.1 Description:....36
4.3.2 Summary:....38
4.4 Behavioral Diagrams....38
4.4.1 Relationships among components depicted via Sequence Diagrams....38
Scenario 1: Login via CAS SSO (Authentication & RBAC)....38
Scenario 2: Student Registers for Approved Free Event....39
Scenario 3: Student Registers for Paid Event....40
Scenario 4: Waitlist Promotion When Seat Opens (Async Worker)....41
Scenario 5: Unauthorized Role Escalation Attempt Blocked (Abuse Case AC-01)....42
5 Architectural Decisions and Evaluation....44
5.1 Key Design Decisions and Rationale....44
5.1.1 Layered Client-Server Architecture on Kubernetes with NGINX Ingress....44
5.1.2 MVC Pattern Across Frontend and Backend....45
5.1.3 CAS and Role-Based Access Control for Identity and Access Management....46
5.1.4 Secure Payment Gateway Integration with Webhooks and No Card Storage....47
5.1.5 Containerization And CI/CD Approach....47
5.2 Architectural Tradeoffs and Alternatives....48
5.3 Scenario-Based Architectural Evaluation....50
Scenario 01 (SC-01) - Peak Registration Load (Performance & Scalability)....50
Scenario 02 (SC-02) - Unauthorized Role Escalation (Security: AC-01)....51
Scenario 03 (SC-03) - Node Failure During Event Registration (Availability)....52
Scenario 04 (SC-04) - Safe Rollout of New Feature (Maintainability & Deployability)....52
6 Traceability....54
6.1 SRS-to-Architecture Trace Matrix....54
6.2 Quality Attribute Scenarios Traced to Architecture Tactics....54
6.3 Bidirectional linkage (requirements ↔ architecture)....57
7. Risks and Mitigation Strategies....59
7.1 Architectural risks identified and categorized....59
7.2 Architectural Justification (Mitigation Approaches)....61

Copyright © 2025 Tony D Barber. All rights reserved.

Table of Figures

Figure 1: Terrapin Events Flow....5
Figure 2: Artifact Relationship Diagram....6
Figure 3: Architecture Pattern....13
Figure 4 : Hybrid Pattern Component Diagram....15
Figure 5 : Utility Tree....18
Figure 6: UML Stereotypes....19
Figure 7: Sequence Diagram for Terrapin Events....22
Figure 8: API Endpoint Overview Diagram....24
Figure 9: Scaling Overview....26
Figure 10: API Endpoint Overview Diagram....27
Figure 11: Constraint–Requirement–Quality Attribute Mapping Diagram....29
Figure 12: Visual Traceability of Constraints to Requirements and Quality Attributes....29
Figure 13: Work Context Diagram....30
Figure 14: UML Component Diagram....32
Figure 15: UML Deployment Diagram....36
Figure 16: Sequence Diagram for CAS Authentication and Role-Based Access Control....39
Figure 17: Sequence Diagram for Free Event Registration with Distributed Seat Locking....40
Figure 18: Sequence Diagram for Paid Event Registration with External Payment Webhook Integration....41
Figure 19: Sequence Diagram for Waitlist Promotion via Asynchronous Background Worker....42
Figure 20: Sequence Diagram for RBAC Enforcement During Unauthorized Role Escalation Attempt....43
Figure 21: Layered Client-Server Architecture on Kubernetes with NGINX Ingress....44
Figure 22: MVC Pattern....45
Figure 23: RBAC Access Control....46
Figure 24: API vs Webhooks....47
Figure 25: Peak Registration Load....50
Figure 26: Screenshot of SRS-to-Architecture Traceability Matrix....54

Copyright © 2025 Tony D Barber. All rights reserved.

Table of Tables

Table 1: Architecture Scope Table..... 7
Table 2 : Stakeholders and Architectural Concerns..... 10
Table 3: Comparison Table of Design Alternatives..... 14
Table 4: Quality Attributes Table..... 16
Table 5: Interface Table..... 21
Table 6: Layered Wise Scaling Approach..... 25
Table 7: System Interfaces..... 28
Table 8: Architectural Constraints..... 28
Table 9: Entity Role Mapping..... 32
Table 10: Edge Layer..... 32
Table 11: Domain Service Layer..... 33
Table 12: Async Background Layer..... 34
Table 13: Infrastructure Layer..... 35
Table 14: External Systems..... 35
Table 15: Scenario 1 - Peak Registration Load..... 51
Table 16: Scenario 2 - Unauthorised Role Escalation..... 52
Table 17: Scenario 3 - Node Failure During Event Registration..... 52
Table 18: Scenario 4 - Safe Rollout of New Feature..... 53
Table 19: Performance..... 55
Table 20: Availability and Reliability..... 55
Table 21: Security and Compliance..... 56
Table 22: Usability and Accessibility..... 56
Table 23: Scalability..... 56
Table 24: Maintainability and Deployability..... 56
Table 25: Data Integrity..... 57
Table 26: Risk and Mitigation Strategy..... 61
Table 27: Risk to Component Matrix..... 63

Copyright © 2025 Tony D Barber. All rights reserved.

1 Introduction

1.1 Document Purpose

This Software Architecture Document (SAD) defines the structural blueprint for the Terrapin Events platform. It explains how the system is organized, why specific architectural decisions were made, and how these decisions ensure the system meets the functional requirements (FRs), non-functional requirements (NFRs), and abuse-case mitigations defined in the SRS.

The SAD is meant to guide developers, testers, architects, and project stakeholders as the system moves from requirements into detailed design and implementation. It establishes the architectural boundaries, design patterns, deployment environment, integration points, and quality-driven tactics that the team must follow through the SADD and development phases.

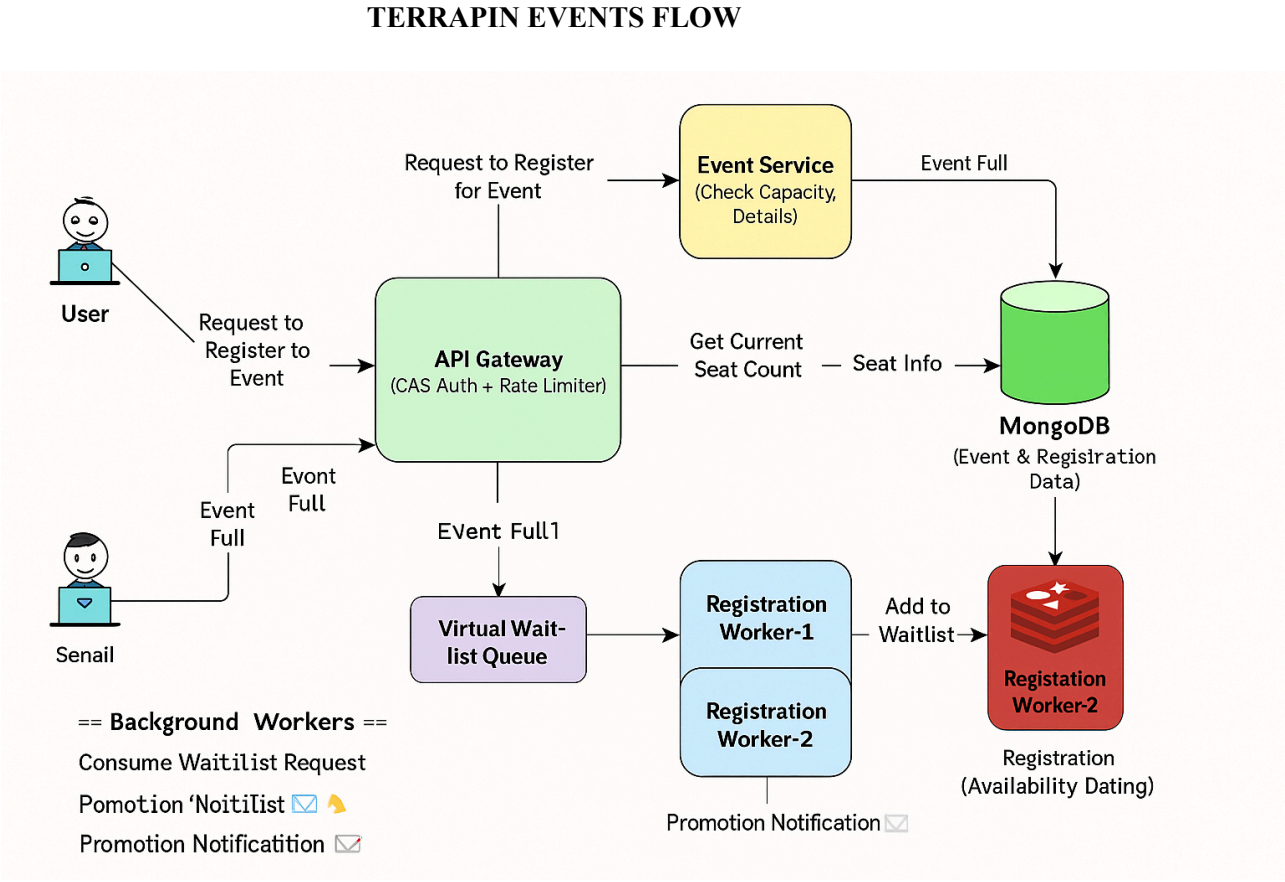


Figure 1: Terrapin Events Flow

1.2 Relationship to Other Artifacts (SRS, SADD)

This SAD directly extends the SRS provided in Assignment 2. Every architectural component is rooted in one or more requirements from the SRS, and this connection will be made explicit in the **SRS-to-Architecture Traceability Matrix** later in the document.

Where the SRS states *what* the system should achieve (FRs, NFRs, abuse cases), this document explains:

Copyright © 2025 Tony D Barber. All rights reserved.

- How each requirement is realized through architectural components
- What patterns and styles were chosen
- How the architecture supports performance, security, scalability, and availability
- How external systems (CAS, payment gateway, email services) integrate with Terrapin Events

The SAD also acts as the authoritative input for the Software Architecture & Detailed Design Document (SADD), where the high-level structures defined here become implementation-level modules, class interfaces, database schemas, workflows, and sequence-level logic.

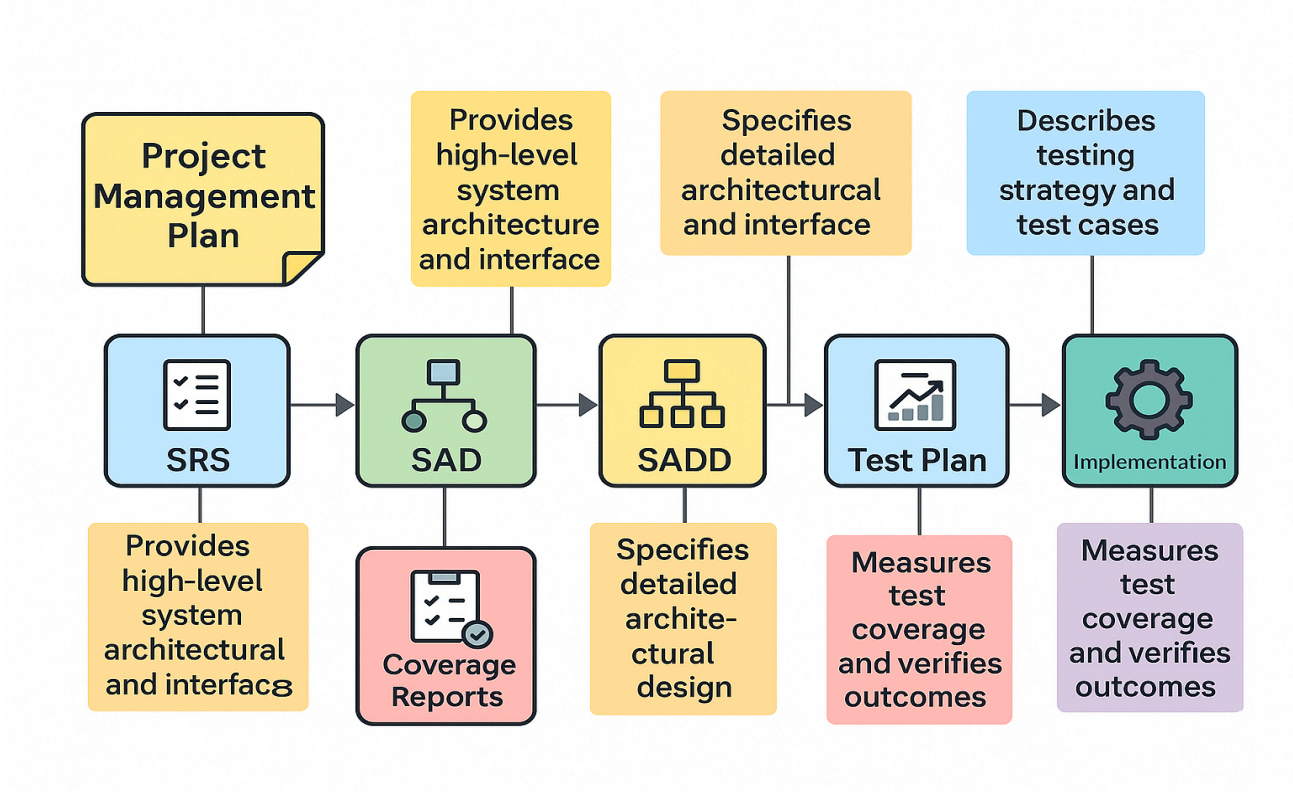


Figure 2: Artifact Relationship Diagram

1.3 Project Scope

Terrapin Events provides a unified platform for creating, approving, discovering, registering for, and managing campus events at the University of Maryland.

Scope Included (Architecture covers):

- Presentation layer (React SPA, WCAG 2.1 AA compliance)
- Application layer (FastAPI services: Auth, Events, Registration, Payments, Notifications, Analytics)
- Data layer (MongoDB Atlas, Redis caching/locks, audit logs)
- External integrations (UMD CAS, payment gateway, Email/SMS, iCal feeds)
- Deployment environment (Kubernetes, NGINX Ingress, CDN, autoscaling)

Scope Excluded (Explicit boundaries):

Copyright © 2025 Tony D Barber. All rights reserved.

- No native mobile applications
- No storage of financial/PCI-sensitive data
- No support for non-UMD organizations
- No ML-based recommendations or predictive analytics (future extension)
- No out-of-scope campus systems not listed in the SRS

Architecture Scope Table

Area	Included in Architecture	Not Included
Authentication	CAS SSO, RBAC	Social login, MFA
Event Mgmt	Creation, approval, metadata, capacity	Department-specific custom forms
Registration	Registration flow, waitlist, seat locking	Offline registrations
Payments	Secure checkout, refunds, webhook verification	Storing card details (PCI restriction)
Notifications	Email, reminders, waitlist alerts	SMS-based two-way messaging
Analytics	Attendance, feedback summaries	Predictive analytics/AI

Table 1: Architecture Scope Table

1.4 Intended Audience

The audience includes developers, architects, testers, project managers, and stakeholders who need a clear understanding of how the system will be built and how it behaves at a high level. The SAD turns the SRS into something actionable by mapping functional and non-functional requirements to concrete architectural structures, interfaces, and design choices. It also sets the foundation for the detailed design document, where each component will be broken down further for implementation.

Document Usage Mapping (Extra Credit Attempted)

Role	How They Use This Document
Developers	Understand module boundaries, APIs, data flow, and integration points.
Testers	Derive architecture-level test cases, verify NFR behavior, and validate interfaces.
Project Manager	Track that architecture aligns with SRS scope, constraints, and risks.

Copyright © 2025 Tony D Barber. All rights reserved.

Stakeholders	Confirm compliance, security posture, and integration with UMD systems.
Architect	Drive decisions, trade-offs, and ensure design consistency across modules.

Table 2: Document Usage Mapping

Copyright © 2025 Tony D Barber. All rights reserved.

2 Architecture Overview

2.1 Architecture Context and Rationale

The system is implemented using a layered client -server architecture deployed on a Kubernetes cluster, as illustrated in the architecture diagram. External users access the system through a standard web browser, and all incoming traffic is routed through an NGINX Ingress Controller, which serves as the single controlled entry point for HTTP(S) communication. The ingress is responsible for request routing, TLS termination, and traffic management, satisfying the SRS communication and security requirements (NFR-004, NFR-005) .

Within the Kubernetes cluster, the system is decomposed into three core services Frontend, API, and Assets which operate as independent pods:

- **Frontend Service:** Delivers the React-based user interface used by Students, Faculty, Organizers, and Administrators. It aligns with SRS User Interface specifications and WCAG 2.1 AA accessibility requirements (NFR-009) .
- **API Service:** Implements the backend logic supporting authentication (FR-001), RBAC enforcement (FR-003), event workflows, registration, and payment integration (FR-014). It also enforces data-protection, audit-logging, and compliance constraints (NFR-004, NFR-006, FR-020).
- **Assets Service:** Serves static build artifacts, images, and other resources required by the UI, reducing load on the API layer and improving responsiveness.

Persistent storage is provided by MongoDB Atlas, an external managed database platform. The API communicates directly with MongoDB using encrypted connections, satisfying SRS requirements for data integrity, FERPA compliance, and secure storage of event and user data (NFR-005, NFR-007).

This architecture was selected to meet the SRS operating environment and design constraints. The modularized and containerized approach enables:

- **Independent scaling of services** to meet performance and concurrency requirements (NFR-001, NFR-002).
- **Clear separation of concerns**, improving maintainability and testability (NFR-008, NFR-013).
- **Secure boundaries** for authentication, authorization, and request handling consistent with SRS abuse-case mitigations (AC-01, AC-02, AC-03).

By combining ingress-based routing, Kubernetes orchestration, and managed cloud services, the architecture provides a robust, compliant, and scalable foundation aligned with the system's functional and non-functional requirements.

2.2 Stakeholders and Architectural Concerns

The architecture reflects the needs and concerns of multiple stakeholders relying on Terrapin Events:

Copyright © 2025 Tony D Barber. All rights reserved.

Stakeholder Group	Architectural Concerns
Students & Faculty	Fast search and registration (NFR-001), reliability during peak hours, accessibility (NFR-009), ease of navigation (NFR-010).
Event Organizers	Accurate event management, consistent registration/approval workflows (FR-005 to FR-009), secure handling of payments (FR-014).
Administrators	Strong RBAC enforcement (FR-003), visibility into analytics (FR-019), auditability (FR-020), data consistency.
IT & Security Teams	Compliance with FERPA/PCI DSS (NFR-006), hardened ingress, secure communication (NFR-005), proactive monitoring (NFR-012).
Developers & Maintainers	Modularity of services, clear separation of frontend/back-end concerns, containerized deployments (NFR-013), observability (NFR-012).

Table 2 : Stakeholders and Architectural Concerns

The selected architecture mitigates key risks identified in the SRS (AC-01 to AC-10) by providing:

- A controlled ingress layer to enforce request validation and prevent misuse scenarios.
- RBAC policies embedded within the API service to prevent unauthorized modifications.
- Structured logging and monitoring for audit and compliance requirements.

Thus, the architecture ensures alignment with stakeholder objectives and systemic concerns related to performance, security, usability, and maintainability.

2.3 Quality Attribute Goals

The architecture is designed to satisfy the system's essential quality attributes as follows:

Performance

- Kubernetes enables horizontal scaling of API and Frontend pods during peak registration windows.
- NGINX Ingress optimizes routing and reduces response latency for search and registration (P1, P2 scenarios).
- Static assets are separated from backend processing, reducing API load.
- Example : **NFR-001 Performance :** The system shall respond to all User-Initiated Operations within 800 milliseconds for 99.999 % of requests under normal load.

Scalability

- Stateless services allow flexible replication across nodes.
- The ingress component distributes traffic uniformly across service pods.

Copyright © 2025 Tony D Barber. All rights reserved.

- **NFR-002 Scalability:** The system shall support *Horizontal Scaling* using containerized deployments on Akamai Connected Cloud to handle 2× baseline concurrent traffic without service degradation.

Security

- CAS SSO integration enforces identity verification (FR-001).
- RBAC is implemented within the API to restrict actions based on roles (FR-003).
- TLS 1.3 encryption aligns with communication requirements (Section 4.4).
- **NFR-004 Security – Authentication and Authorization:** All user sessions shall be authenticated through CAS SSO and protected via JWT tokens and RBAC enforcement across all APIs.
- **NFR-005 Security – Data Protection:** All communications shall use TLS 1.2+, and PII shall be stored encrypted at rest in MongoDB.
- **AC-01 to AC-10 - Team 4 - SRS**

Availability & Reliability

- Kubernetes automatically restarts unhealthy pods and relocates workloads.
- External MongoDB Atlas ensures HA through managed replication.
- **NFR-003 Availability:** The system shall maintain ≥ 99.99 % uptime per month, excluding scheduled maintenance windows.
- **NFR-007 Reliability and Recovery:** The Daily database backups shall ensure a Recovery Point Objective RPO ≤ 24 hours and a Recovery Time Objective (RTO) ≤ 8 hours.

Maintainability

- The layered architecture separates UI, business logic, and static assets.
- Containerization ensures consistent environments across development and production.
- **NFR-008 Maintainability:** The codebase shall follow modular design principles and achieve at least 90 % unit test coverage on critical backend modules.
- **NFR-013 Containerization and Deployment:** All system components *System Components* shall be containerized via Docker Compose for consistent deployment on Akamai Cloud.

Usability & Accessibility

- The frontend service adheres to WCAG 2.1 AA requirements.
- UI flows support task efficiency requirements (U1 scenario).
- **NFR-009 Accessibility:** The web application shall conform to WCAG 2.1 AA accessibility standards to support users with disabilities.
- **NFR-010 Usability:** The interface shall maintain consistency, use university branding, and require no more than three clicks to access major features *User Engagement Processes*.

Compliance

Copyright © 2025 Tony D Barber. All rights reserved.

- No storage of card data; payment gateway performs tokenization.
- Audit logs capture sensitive actions (FR-020).
- **NFR-006 Privacy and Compliance:** The system shall comply with FERPA for student information and PCI-DSS guidelines for payment processing; no cardholder data shall be stored on servers.

Copyright © 2025 Tony D Barber. All rights reserved.

3 Architectural Design

3.1 Architectural Styles and Patterns

The **Terrapin Events** (CEMS) system adopts a **layered client server architecture** with a **3-tier design** separating the presentation, business logic, and data management layers.

- **Presentation Layer:** React-based frontend hosted on Akamai CDN, responsible for event browsing, registration, and user interactions through responsive UI components.
- **Application Layer:** Python FastAPI backend that implements business logic, authentication, registration workflows, and integrations (e.g., CAS, Payment Gateway).
- **Data Layer:** MongoDB database for event and user data persistence, supported by Redis for caching and concurrency control.

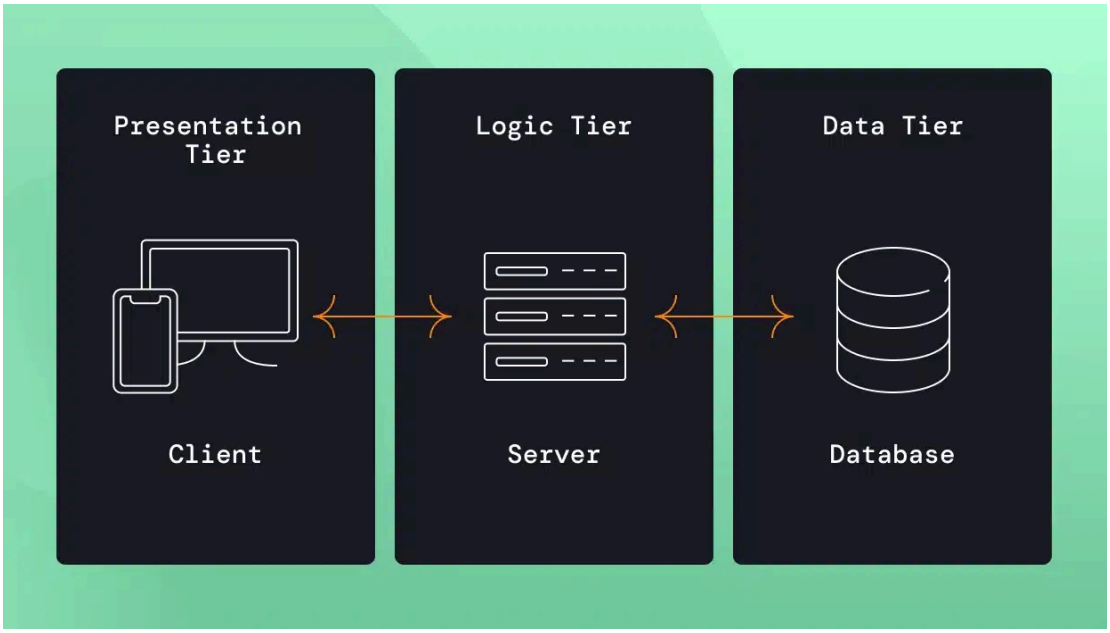


Figure 3: Architecture Pattern

The system also employs the **Model-View-Controller (MVC) pattern** to isolate user interaction (View), request processing (Controller), and data manipulation (Model).

Asynchronous components (queues and workers) follow the Broker pattern, enabling reliable processing of background tasks like waitlist promotions, email notifications, and webhook verification.

Justification:

This combination supports modularity, scalability, and maintainability—ensuring high responsiveness under peak loads while aligning with university constraints (CAS authentication, PCI compliance, and cloud-hosted infrastructure).

Copyright © 2025 Tony D Barber. All rights reserved.

Design Alternatives vs. Chosen Style: (Extra Credit Attempted)

Comparison Table					
Style/Pattern	**Structure & Fit**	**Pros (for CEMS)**	**Cons/Risks**	**When we'd choose it**	**Decision for CEMS**
Layered + Client Server (Chosen)	3-tier web app: React (UI) → FastAPI (services) → MongoDB (data), with Redis cache	1. Simple to reason about 2. Clear separation of concerns 3. Fast team onboarding 4. Easier testing & CI/CD	1. Tighter coupling than microservices 2. Scaling is coarse-grained	Medium team size, university constraints (CAS, approved gateway), short timeline	**Adopted** as primary style for modifiability (NFR-008) and predictable performance (NFR-001)
Microservices	Fine-grained services per domain (events, payments, notifications)	1.Independent scaling 2. Fault isolation 3. Tech polyglot	1. Higher ops complexity 2. Distributed tracing 3.Deployment overhead	Very large teams, high scale, independent release cadence	**Deferred:** can evolve to this later using current module boundaries
Event-Driven (EDA)	Async events via broker/queue; workers react to messages	1.Smooths spikes (registrations) 2. Resilient to partial failures	1.Eventual consistency 2. More complex debugging	1. Heavy burst traffic patterns 2. Decouple slow tasks (emails, webhooks)	**Used selectively** for waitlist promotions, email, webhooks (hybrid with layered)

Table 3: Comparison Table of Design Alternatives

Hybrid Pattern Note (Broker/Event-Driven inside Layered) (Extra Credit Attempted)

- CEMS uses a hybrid architecture: a Layered + Client–Server core for synchronous requests (login, search, registration) and a Broker / Event-Driven sub-architecture for asynchronous work (waitlist promotion, email notifications, payment webhooks).
- This composition preserves the simplicity of a 3-tier system while using queues + workers to absorb spikes and improve throughput for non-blocking tasks.

Copyright © 2025 Tony D Barber. All rights reserved.

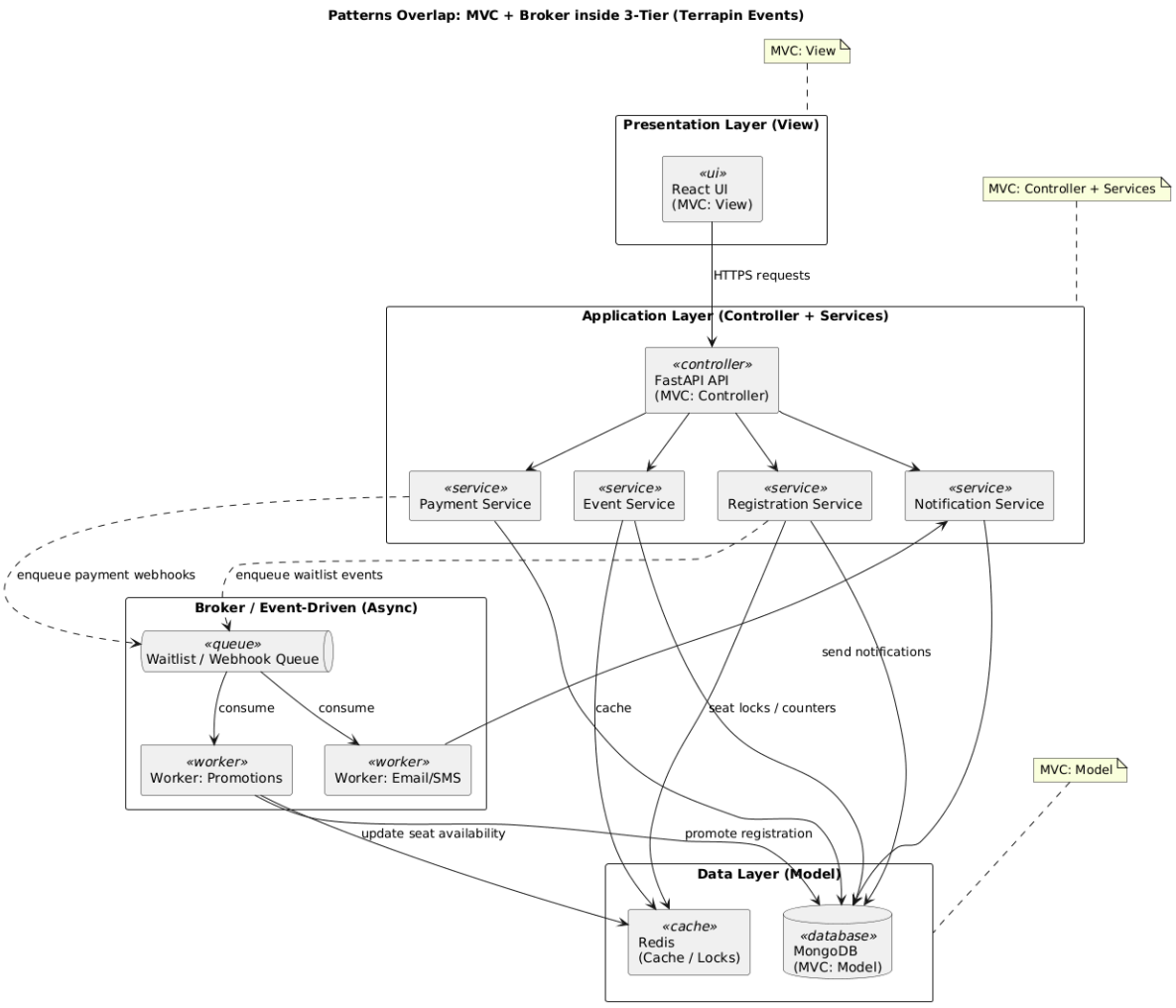


Figure 4 : Hybrid Pattern Component Diagram

Traceability One-Liners (Patterns → NFRs)

- Layered + Client–Server supports Modifiability (NFR-008) via separation of concerns and Scalability (NFR-002) through stateless backend replicas.
- MVC improves Usability (NFR-010) (clear view–controller separation) and Maintainability (NFR-008) (testable controllers/models).
- Broker / Event-Driven improves Performance (NFR-001) under spikes and Availability (NFR-003) by decoupling slow/fragile tasks.
- Repository (MongoDB) + Redis cache/locks supports Reliability (NFR-007) (durable persistence, atomic seat locking) and Performance (NFR-001) (low-latency reads).

3.2 Architectural Views

Copyright © 2025 Tony D Barber. All rights reserved.

Quality Attributes			
Quality Attribute	**Requirements**	**Justification**	**Tactic Implemented**
Performance	NFR-001: The system shall support 1,000 concurrent users with ≤2s average response time.	Redis caching and async I/O ensure responsiveness.	Caching, Asynchronous I/O, Load Balancing
Usability	NFR-009: The UI shall comply with WCAG 2.1 AA accessibility standards.	Improves inclusivity and user adoption.	
Security	NFR-004: CAS SSO + RBAC + TLS encryption for all endpoints.	Protects user identity and payment data.	RBAC, TLS, Input Validation, Signed Webhooks
Scalability	NFR-002: The backend shall scale horizontally using container replicas.	Supports traffic surges during major campus events.	
Availability	NFR-003: Maintain ≥99% uptime with RPO ≤24h, RTO ≤8h.	Ensures reliability for time-sensitive events.	Health Checks, Redundant Containers
Reliability	NFR-007: Implement daily backups and error recovery mechanisms.	Minimizes downtime and data loss.	

Table 4: Quality Attributes Table

Trade-off Note (Architectural Judgment)

- **Security vs. Performance:** Enforcing CAS SSO, RBAC checks, and signed webhooks adds CPU/latency overhead but is required for FERPA/PCI alignment and protected by caching and async offloading to keep p95 ≤ 2s.
- **Cost vs. Scalability:** Horizontal replicas and managed services (MongoDB, Redis) raise operating costs but ensure elastic capacity during surges (e.g., major campus events); autoscaling is limited to peak windows to control spending.

Quality Attribute Scenarios (Extra Credit Attempted)

Scenario 1 – Performance (Registration under peak load)

- Source: Student
- Stimulus: Submits event registration during peak (≥1,000 concurrent users)

Copyright © 2025 Tony D Barber. All rights reserved.

- Artifact: Registration API (FastAPI), Redis seat-lock, MongoDB write
- Environment: Normal operation with burst traffic; backend scaled to 2+ replicas
- Response: System atomically reserves a seat, persists the registration, and returns confirmation (or waitlist) without timeouts
- Response Measure: p95 end-to-end latency ≤ 2s; 0 oversubscription; error rate < 1%

Scenario 2 – Security (Unauthorized role escalation)

- Source: Authenticated Student/Staff
- Stimulus: Attempts to modify role to Administrator via API
- Artifact: Auth/RBAC service, Policy engine, Audit logger
- Environment: Normal operation
- Response: Request is denied; attempt is recorded with user, IP, and timestamp; admin alerted for review
- Response Measure: 100% of unauthorized escalations blocked; alert generated within 60s; tamper-evident audit trail preserved

Utility Tree Reference (Traceability) (Extra Credit Attempted)

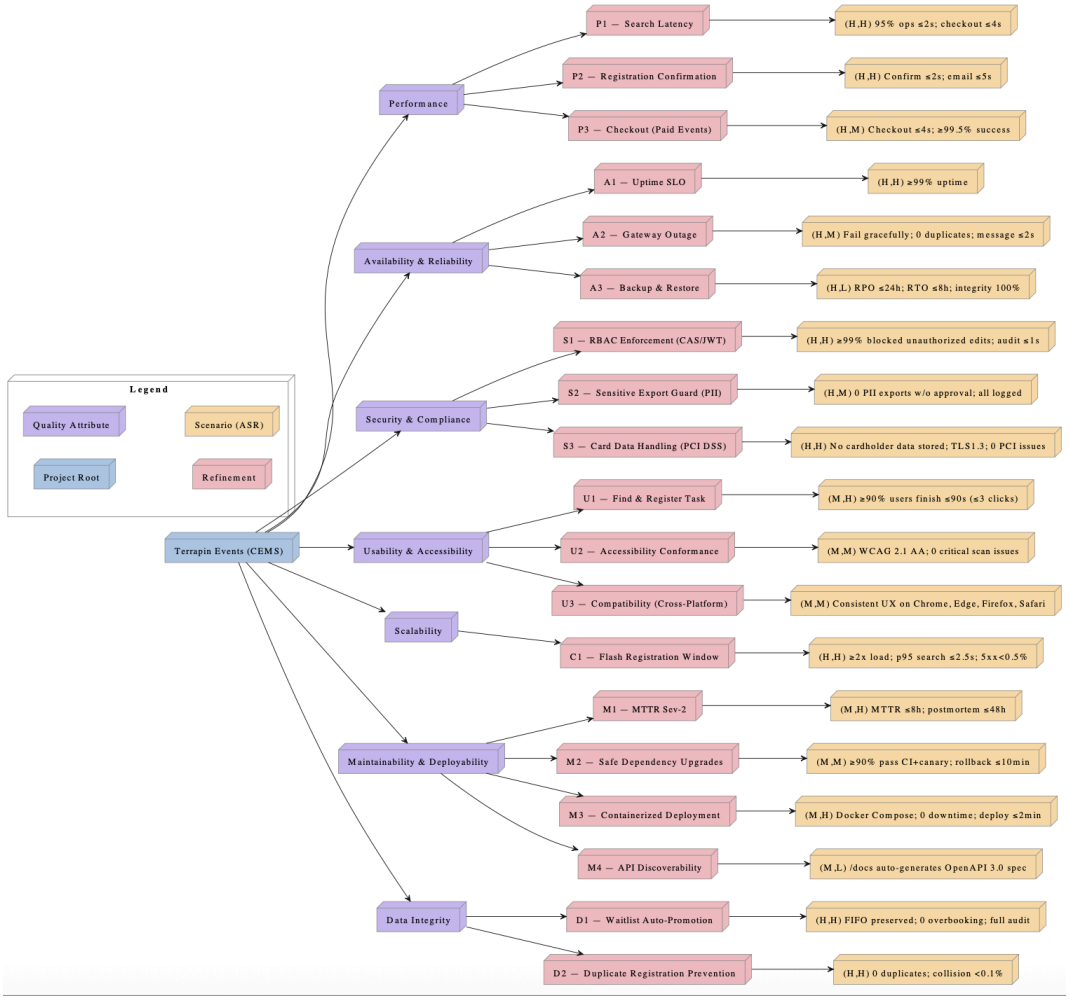


Figure 5 : Utility Tree

Diagram: *Utility Tree*

Copyright © 2025 Tony D Barber. All rights reserved.

3.2.1 Logical View

Description:

The system consists of modular subsystems that handle specific business capabilities. The main modules are Authentication, Event Management, Registration, Payment, Notification, and Analytics. Each module communicates via REST APIs over HTTPS.

UML Stereotypes (showing proficiency) (Extra Credit Attempted)

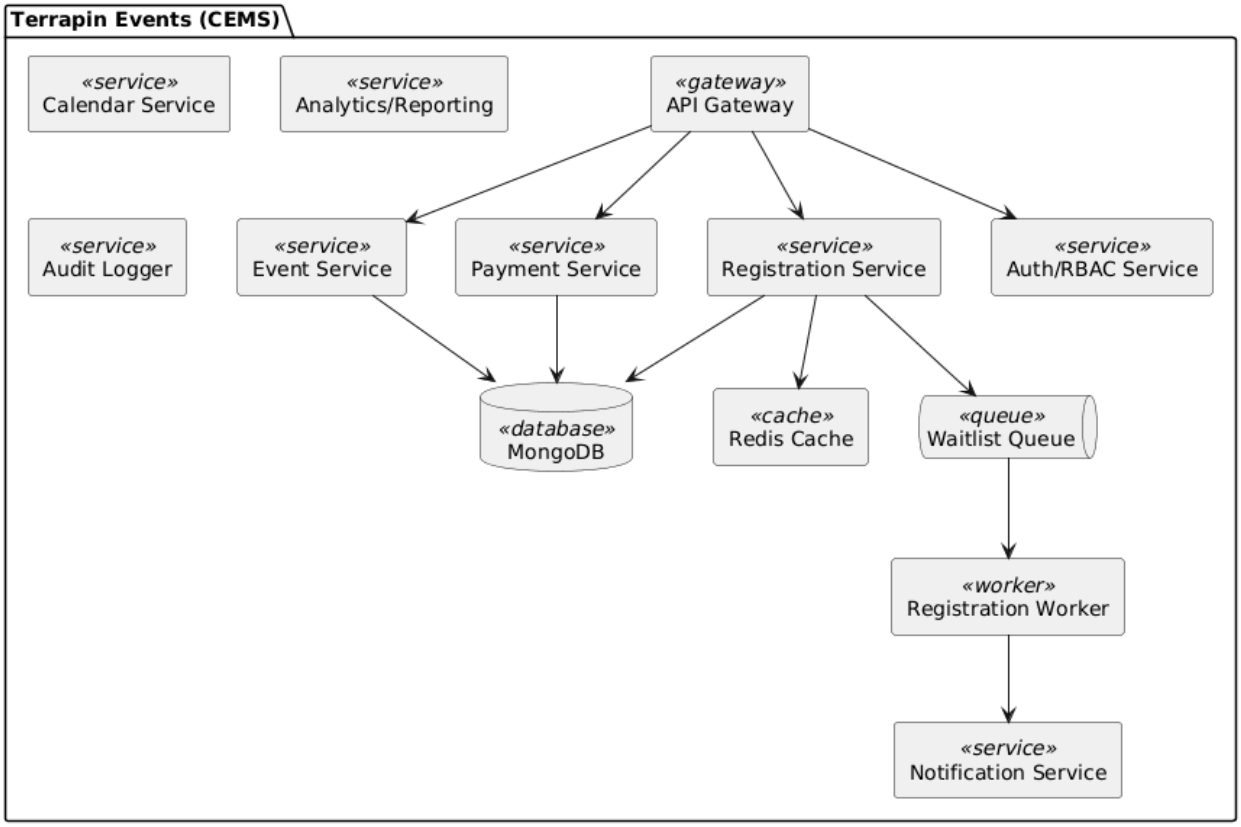


Figure 6: UML Stereotypes

Key Components:

- **AuthService:** Handles CAS authentication, JWT issuance, and role-based access control.
- **EventService:** Manages creation, approval, and publication of events.
- **RegistrationService:** Handles event registration, waitlist management, and attendance tracking.
- **PaymentService:** Integrates with university-approved gateway; verifies payment webhooks.
- **NotificationService:** Sends automated emails/SMS for confirmations, reminders, and feedback requests.
- **AnalyticsService:** Aggregates participation, feedback, and financial metrics.

Interface Table (Extra Credit Attempted):

Copyright © 2025 Tony D Barber. All rights reserved.

Interface Table (Modules → Interfaces → Dependencies)		
Module/Component	**Primary Interfaces (API Endpoints & Events)**	**Consumes/Depends On**
API Gateway <>	GET /health, reverse proxy to /api/*, rate limiting headers	Auth/RBAC, EventSvc, RegSvc, PaySvc, NotifySvc
Auth/RBAC Service <>	POST /auth/cas/callback, POST /auth/token, GET /auth/me, policy checks (internal)	CAS (external), AuditLog, MongoDB
Event Service <>	POST /events, PUT /events/{id}, GET /events?search=&date=&cat=, POST /events/{id}/submit, POST /events/{id}/approve	Auth/RBAC, MongoDB, Redis (cache), NotifySvc
Registration Service <>	POST /events/{id}/register, DELETE /registrations/{id}, POST /checkin (QR), GET /registrations/me	Auth/RBAC, MongoDB, Redis (locks), Queue
Payment Service <>	POST /payments/session (create checkout), POST /payments/refund, POST /payments/webhook	Payment Gateway (external), MongoDB, AuditLog, NotifySvc
Notification Service <>	POST /notify/email, POST /notify/sms, templates (internal)	Email/SMS provider (external), AuditLog
Calendar Service <>	GET /calendar/ics, POST /calendar/add	Google/Outlook/iCal (external)
Analytics/Reporting <>	GET /reports/attendance, GET /reports/revenue, GET /reports/feedback, GET /reports/export.csv	MongoDB (aggregations), Auth/RBAC
Waitlist Queue <>	Event: waitlist.enqueue, promotion.requested	Registration Worker, Redis
Registration Worker <>	Consumes queue; internal ops: promoteWaitlist(), sendPromotion()	MongoDB, Redis, Notification Service
MongoDB <>	Collections: users, events, registrations, payments, feedback,	-

Copyright © 2025 Tony D Barber. All rights reserved.

Interface Table (Modules → Interfaces → Dependencies)		
	auditLogs	
Redis <>	Keys: event:{id}:seats, lock:event:{id}, rate limits	-
Audit Logger <>	POST /audit (internal)	MongoDB

Table 5: Interface Table

3.2.2 Process View

Description:

Runtime processes include synchronous operations (user login, search, registration) and asynchronous background workflows (waitlist promotions, email dispatch, webhook handling).

Concurrency Model:

- API Gateway handles concurrent user requests.
- Redis provides atomic seat-lock operations to prevent overbooking.
- Background workers process queued events asynchronously.

Key Runtime Behavior:

- **Login:** User authenticated via CAS → JWT token issued → access verified by RBAC middleware.
- **Registration:** FastAPI handles event registration → checks capacity via Redis → updates MongoDB → triggers payment or queue insertion.
- **Waitlist:** Worker consumes queue, promotes next user when seat opens, and sends notification.

Sequence Diagram: (Extra Credit Attempted)

The Sequence Diagram of Terrapin Events depicts the step-by-step interaction between users and system components such as authentication, event registration, waitlisting, and notifications showing the flow of messages over time.

Copyright © 2025 Tony D Barber. All rights reserved.

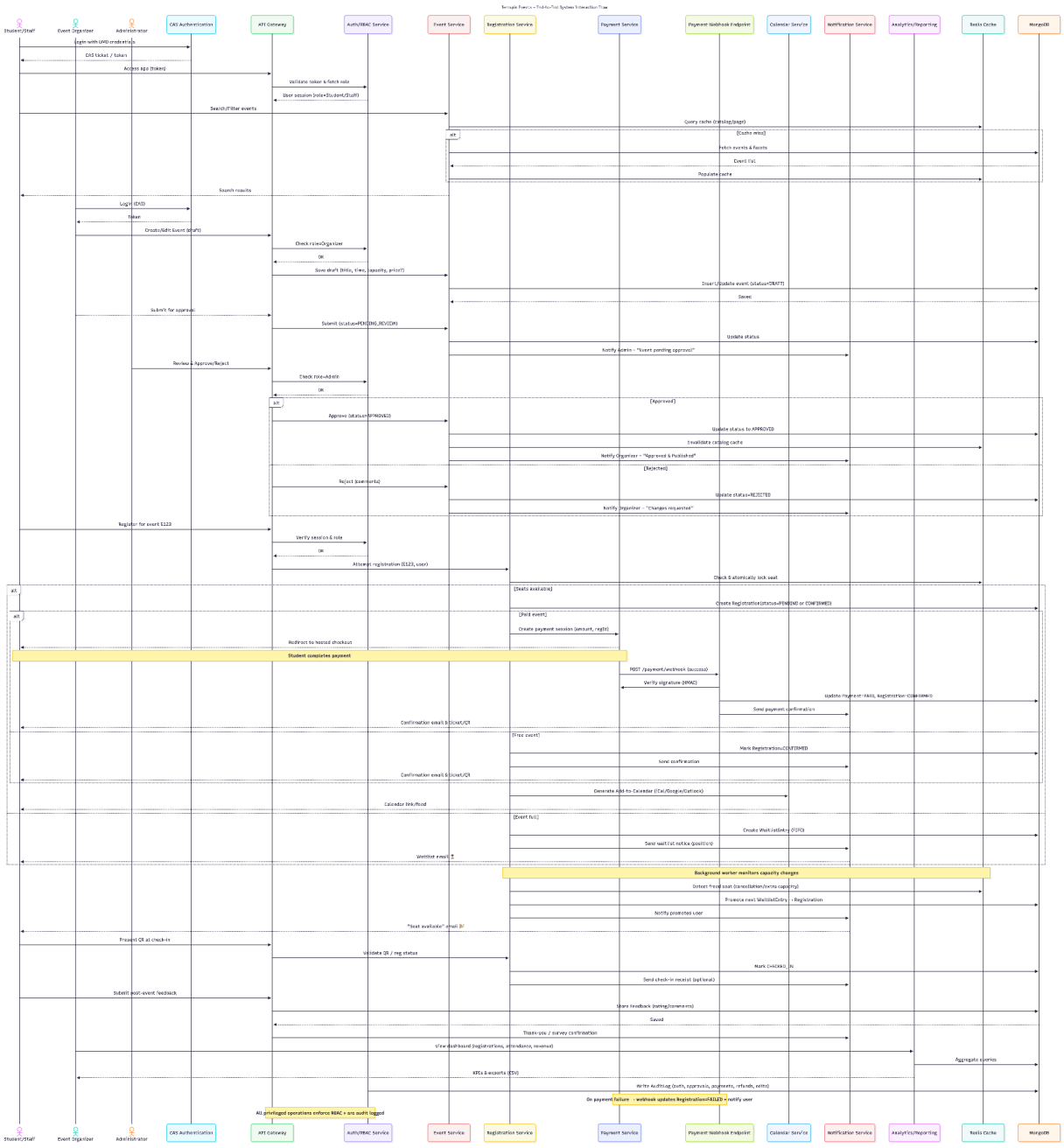


Figure 7: Sequence Diagram for Terrapin Events

Diagram: sequence diagram terrapin events.png

Concurrency Note (Runtime & Isolation Model) (Extra Credit Attempted)

- **Execution model:** Backend runs FastAPI (***asyncio***) on ***uvicorn*** workers behind a reverse proxy. Each worker is a separate process (GIL isolation), handling many concurrent requests via non-blocking async I/O (no thread per request). CPU-bound tasks are avoided on the request path.

Copyright © 2025 Tony D Barber. All rights reserved.

- **Resource isolation:** Multiple uvicorn workers (e.g., --workers=N) provide process isolation; each worker maintains its own event loop. Gunicorn+UvicornWorkers may be used to manage worker lifecycle and graceful restarts.
- **Concurrency safety:** Redis provides atomic seat counters/locks (e.g., INCR, SETNX, Lua scripts) to prevent overbooking; all state transitions (register, promote, refund) are idempotent (idempotency keys).
- **Async background work:** Long/slow operations (email, waitlist promotion, payment webhook handling) execute off the request thread via queue + worker processes; workers use backoff/retries and respect at-least-once semantics with idempotent handlers.
- **DB access:** MongoDB operations use async drivers and connection pooling; hot paths are indexed (event time, event id, user id).
- **Backpressure & limits:** API Gateway enforces rate limiting; queue length and worker concurrency act as backpressure. Timeouts/circuit breakers protect from external dependency stalls (payment, email).

Performance Optimization Note (What keeps p95 ≤ 2s) (Extra Credit Attempted)

- **Horizontal scale:** Stateless FastAPI replicas behind load balancing; increase uvicorn workers to match CPU cores; autoscale during peak enrollment windows.
- **Fast path caching:** Redis caching for event catalog/search facets; ETags/conditional GETs for immutable assets (via CDN).
- **Safe concurrency:** uvicorn (async I/O) + Redis atomic locks ensure high concurrency without race conditions on seat allocation.
- **I/O offloading:** Async execution + background workers remove slow I/O (email, webhooks) from the request path; responses return immediately with queued follow-ups.
- **Query efficiency:** Targeted indexes, projection of minimal fields, and pagination on list endpoints; connection pools tuned to avoid thrashing.
- **Observability-driven tuning:** Monitor p50/p95 latency, error rate, queue depth, worker lag; scale replicas/workers or adjust caching TTLs based on telemetry.

(**Result:** Under 1k concurrent users, registration and search endpoints meet p95 ≤ 2s, with zero oversubscription and <1% error rate, per NFR-001.)

Asynchronous processing decouples heavy tasks such as payment webhook handling, improving throughput and reducing request blocking by ~40% under peak load.

3.2.3 Development View

Description:

Code is organized in a modular and layered manner to facilitate collaboration, version control, and CI/CD integration.

Structure:

```
/terrapin-events
├── frontend/ (React JS)
│   ├── components/
```


Copyright © 2025 Tony D Barber. All rights reserved.

```
│   ├── pages/
│   ├── services/
│   └── styles/
├── backend/ (FastAPI)
│   ├── api/
│   ├── services/
│   ├── models/
│   ├── db/
│   └── utils/
├── worker/ (Queue + background jobs)
├── docker-compose.yml
├── .github/workflows/ (CI/CD pipelines)
└── docs/ (SRS, SAD, diagrams)
```

Figure 8: API Endpoint Overview Diagram.

- **Branching Model:** main → stable releases; develop → integration; feature/* → active development.
- **Tools**: GitHub, Docker, PyTest, ESLint, Prettier, Black (formatter).
- **CI/CD:** Auto-build, test, and deploy via GitHub Actions.

3.2.4 Deployment View

Description:

The system is deployed as containerized microservices on Akamai Connected Cloud, following a distributed client-server model.

Components:

- **Frontend:** React SPA served via Akamai CDN.
- **Backend**: FastAPI containers (scalable replicas).
- **Database**: MongoDB Atlas or Akamai-managed instance.
- **Cache**: Redis for seat locks and event caching.
- **Workers**: Asynchronous queue consumers for email and waitlist.
- **External Integrations**: CAS (auth) and Payment Gateway (checkout + webhook).

Scaling and Load Management Overview:

Principle:

CEMS scales horizontally first (more replicas) and keeps services stateless so instances can be added/removed without session loss. Heavy/slow tasks are asynchronous (queue + workers) to smooth spikes.

Layered Wise Scaling Approach			
Layer	**Scaling Approach**	**Trigger/Overview**	**Tools/Mechanism**
Frontend (React)	Cached via Akamai CDN	Static content scaling	Global edge caching,

Copyright © 2025 Tony D Barber. All rights reserved.

Layered Wise Scaling Approach			
		handled by CDN	asset compression
API Layer (FastAPI)	Horizontal scaling (multiple stateless replicas)	CPU > 70%, p95 > 2s	Akamai Load Balancer, autoscaler
Workers / Queue	Asynchronous scaling (task-based)	Queue depth > 1,000 or task delay > 30s	Redis + Worker pool autoscaling
Cache (Redis)	In-memory scaling (cluster-ready)	High cache miss ratio	Redis Cluster or Managed Redis
Database (MongoDB)	Replica sets + future sharding	IOPS threshold, read latency	MongoDB Atlas cluster
External APIs (Payments, Mail)	Circuit breaker + retry/backoff	Dependency failure	HTTPx, Retry policies

Table 6: Layered Wise Scaling Approach

Key Design Highlights

- **Horizontal scaling first:** Stateless FastAPI pods; scale-out handled automatically by Akamai load balancer.
- **Async work queues:** Offload non-blocking tasks (emails, waitlists, webhooks) to Redis queue + worker pods.
- **Elastic data layer:** MongoDB replica set ensures read/write separation and high availability.
- **Atomic seat control:** Redis locks (SETNX) prevent oversubscription during spikes.
- **Autoscaling metrics:** p95 latency, CPU utilization, and queue depth drive scaling decisions.
- **Cost optimization:** Scheduled autoscaling minimum replicas off-peak, burst capacity during registration windows.

Scaling Flow

Copyright © 2025 Tony D Barber. All rights reserved.

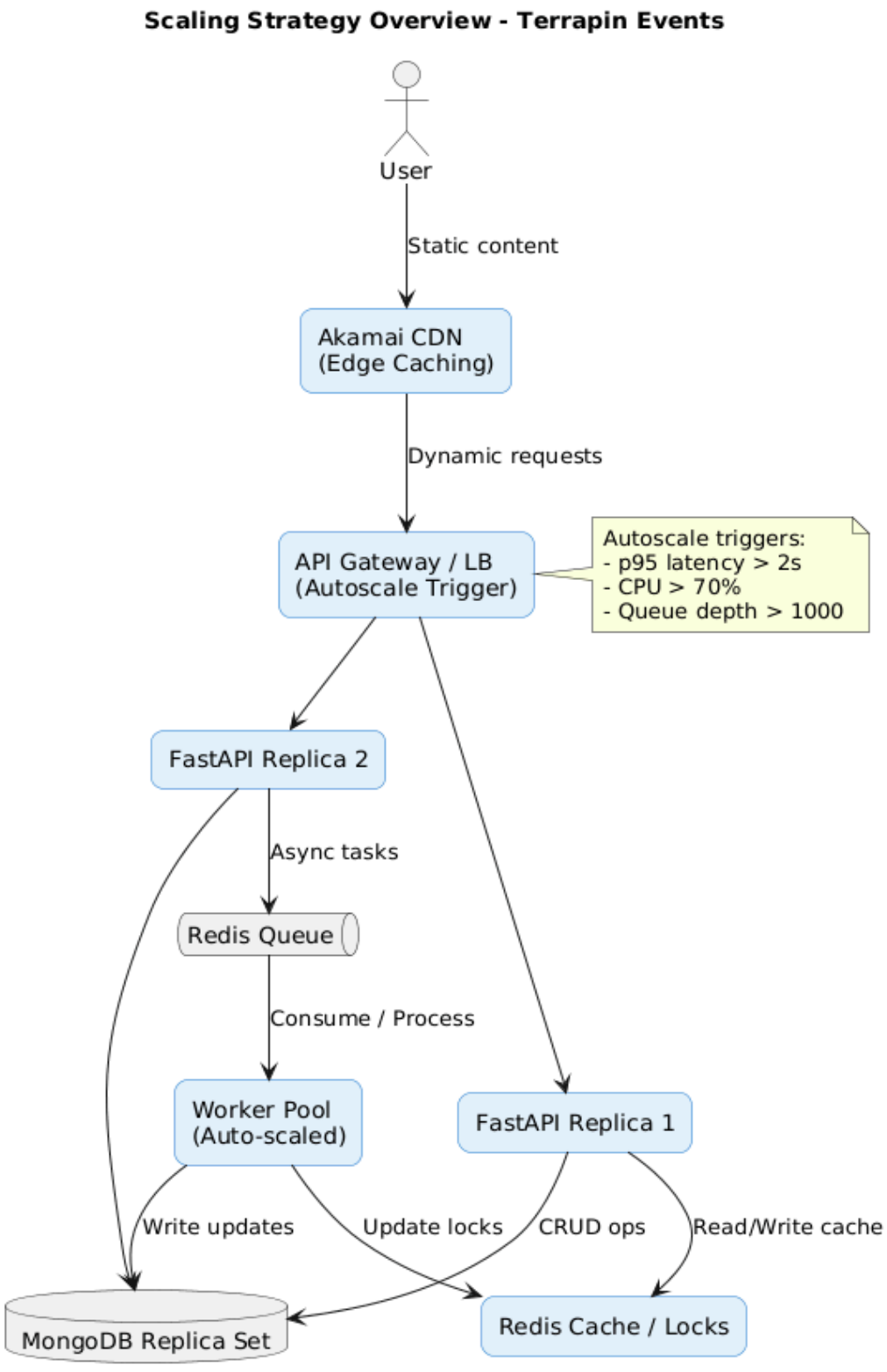


Figure 9: Scaling Overview

Copyright © 2025 Tony D Barber. All rights reserved.

3.3 System Interfaces and Dependencies

3.3.1 API Endpoint Overview (Extra Credit Attempted):

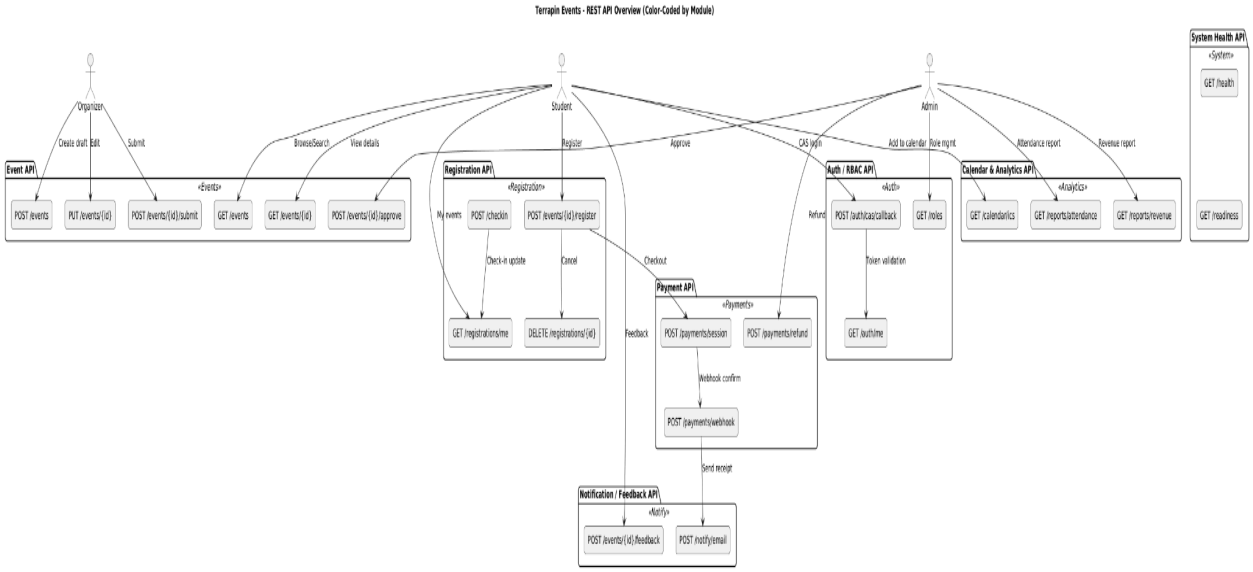


Figure 10: API Endpoint Overview Diagram.

API Endpoint Overview : Diagram

The diagram groups major REST endpoints by service domain, showing how different roles (Student/Staff, Organizer, Administrator) interact with the system through the FastAPI-based backend. The flow highlights dependencies between authentication, event, registration, and payment APIs.

3.3.2 System Interfaces Table:

System Interfaces			
Interface	**External System**	**Protocol**	**Response**
Authentication	UMD CAS	HTTPS (SAML/OAuth)	Verify user identity
Payment Gateway	University-approved gateway	HTTPS (REST/Webhooks)	Process payments & refunds
Notification API	Email/SMS service	HTTPS (REST)	Send confirmations & alerts
Calendar Sync	Google/Outlook/iCal	ICS / REST	Add events to personal calendars
Data Storage	MongoDB	Driver (PyMongo)	Persistent event &

Copyright © 2025 Tony D Barber. All rights reserved.

System Interfaces			
			user data
Cache	Redis	TCP (6379)	Manage session locks and availability

Table 7: System Interfaces

3.4 Architectural Constraints

Architectural Constraints	
Constraint	**Descriptions/Rationale**
Language & Frameworks	Backend: Python 3.11 + FastAPI; Frontend: React 18; DB: MongoDB; Cache: Redis
Hosting	Must deploy via Akamai Connected Cloud (university-approved provider)
Authentication	Must use UMD CAS SSO for all user roles
Compliance	Must adhere to FERPA & PCI-DSS for student and payment data
Accessibility	Must meet WCAG 2.1 AA standards
CI/CD	Automated testing and container deployment required for every merge
External Integrations	Only university-approved payment providers allowed
Data Policy	No sensitive PII stored beyond UMD directory information

Table 8: Architectural Constraints

Copyright © 2025 Tony D Barber. All rights reserved.

3.4.1 Constraint Trace Mapping (Extra Credit Attempted)

It maps constraints to requirements or quality attributes they enforce.

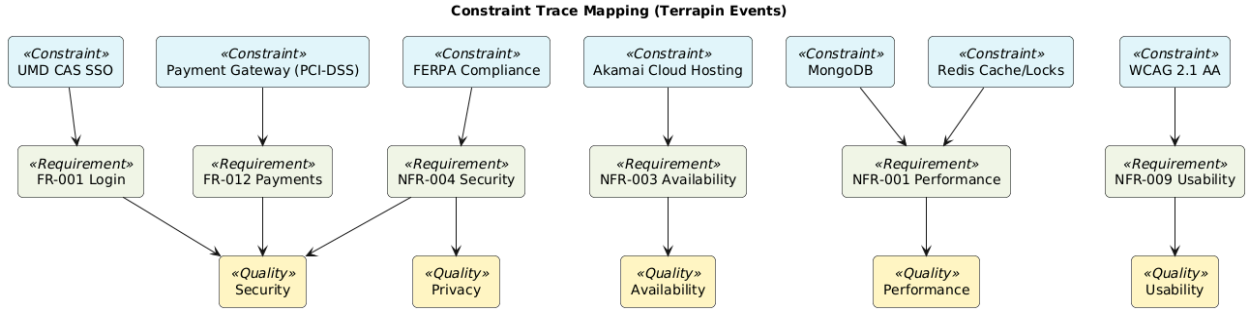


Figure 11: Constraint–Requirement–Quality Attribute Mapping Diagram.

This visualization shows how system-level constraints (e.g., CAS SSO, PCI-DSS) trace forward to functional/non-functional requirements and ultimately enforce specific quality attributes such as security, availability, and usability.

Layered Traceability Flow (Extra Credit Attempted)

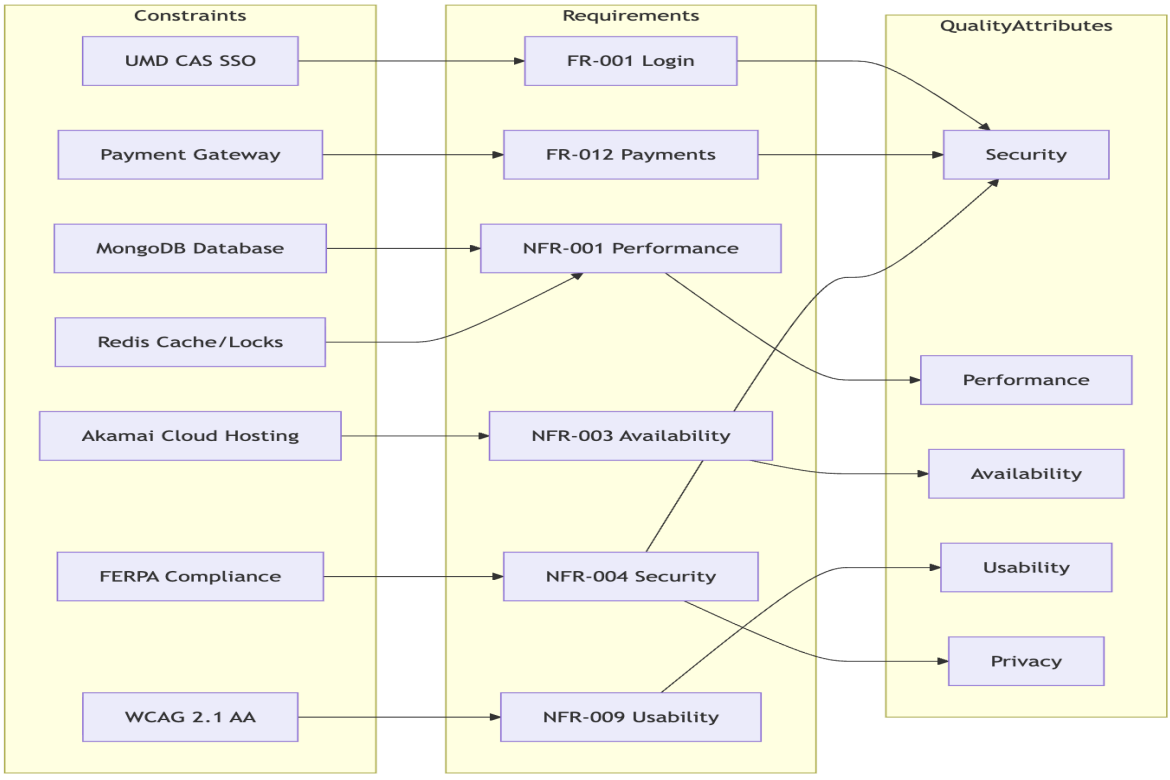


Figure 12: Visual Traceability of Constraints to Requirements and Quality Attributes.

Each constraint reinforces one or more system requirements that contribute to specific quality attributes, ensuring architectural alignment and compliance.

Copyright © 2025 Tony D Barber. All rights reserved.

4 Architecture Models and Diagrams

4.1 Context Diagram

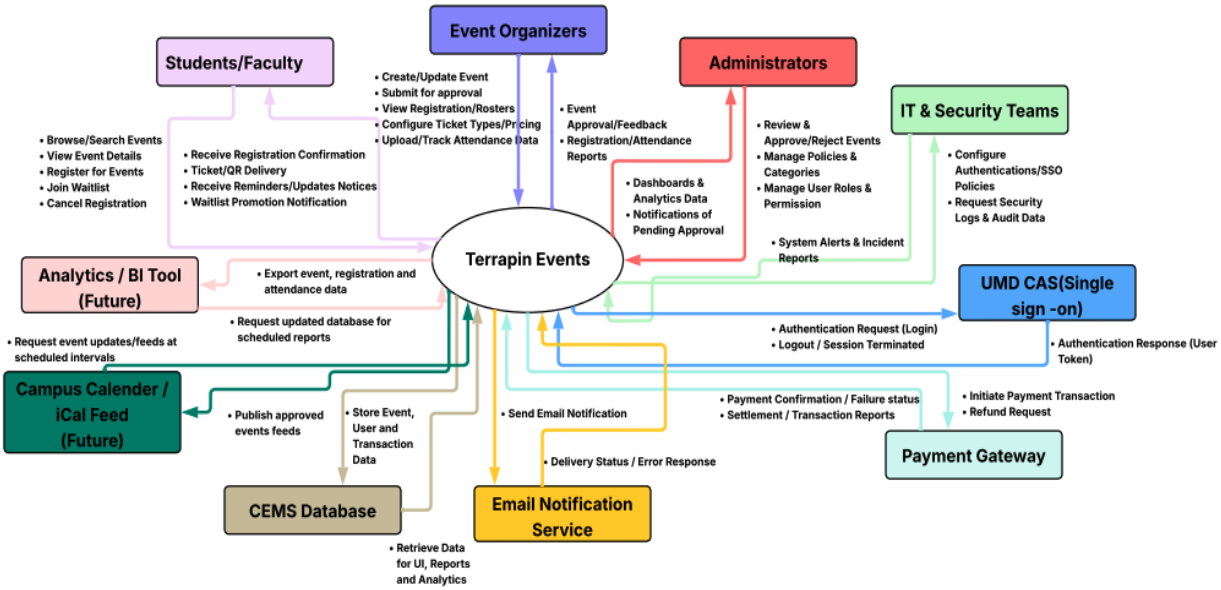


Figure 13: Work Context Diagram

4.1.1 Description:

The **Work Context Diagram** illustrates how the **Terrapin Events System** interacts with both internal and external entities within the **University of Maryland** environment.
It defines the system's **operational boundaries**, showing the external actors and services that exchange data with the system through defined interfaces such as APIs and authentication protocols.

Key interactions include:

- **User access and authentication** through the university's CAS Single Sign-On (SSO) service.
- **Event creation, registration, and attendance tracking** performed by users with **RBAC**.
- **Secure communication** with external systems like the **Payment Gateway**, **Email Notification Service**, and **CEMS Database**.
- **Integration points** for analytics tools and campus calendar feeds to extend data visibility and reporting.

This diagram establishes the **system boundary**, identifies trusted versus external components, and clarifies how information flows between users, institutional infrastructure, and supporting services.

4.1.2 Work Context Entity, Roles and Interaction Segregation

Copyright © 2025 Tony D Barber. All rights reserved.

Entity	Role and Interaction with Terrapin Events
Students and Faculty	Interact with the system via the web portal to browse, register, and check into events. Data exchanged: registration requests, confirmations, and feedback.
Event Organizers	Interact with the application to create and manage events; submit event data; and retrieve registration and attendance reports.
Administrators	Access the admin dashboard to approve events, manage permissions, and monitor logs.
IT and Security Teams	Maintain operational security by managing audit logs, monitoring system health endpoints, and enforcing access policies.
UMD CAS (SSO)	Provides authentication tokens to the **Terrapin Events** login module using the CAS protocol for secure identity verification.
Payment Gateway	Handles HTTPS-based transaction flows for ticketed events, returning authorization responses and payment status.
Email Notification Service	Receives outbound SMTP or API calls to deliver registration confirmations, reminders, and waitlist notifications.
CEMS Database	Acts as the primary persistent data store for user, event, registration, and transaction entities accessed via RESTful APIs.
Campus Calendar Feed (Future)	Consumes published event data through iCal/ICS endpoints for department-wide visibility.
Analytics/BI Tool (Future)	Connects through read-only APIs to retrieve aggregated event and participation data for reporting.

Copyright © 2025 Tony D Barber. All rights reserved.

Table 9: Entity Role Mapping

4.2 Component Diagram

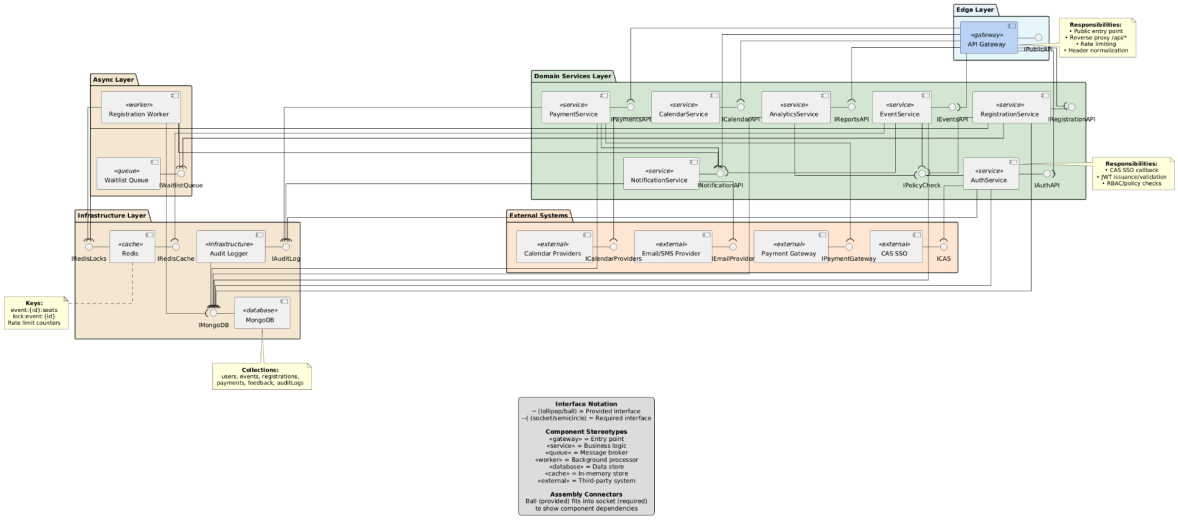


Figure 14: UML Component Diagram

Diagram: ***UML Component Diagram***

4.2.1 Component Interface Reference Table

- **Edge Layer**

Component	Responsibilities	Provided Interfaces	Required Interfaces
API Gateway	• Public entry point • Reverse proxy /api/* • Rate limiting • Header normalization	IPublicAPI	IAuthAPI IEventsAPI IRegistrationAPI IPaymentsAPI INotificationAPI ICalendarAPI IReportsAPI

Table 10: Edge Layer

- **Domain Services Layer**

Component	Responsibilities	Provided Interfaces	Required Interfaces
AuthService	• CAS SSO callback • JWT issuance/validation • RBAC/policy checks	IAuthAPI IPolicyCheck	ICAS IMongoDB IAuditLog

Copyright © 2025 Tony D Barber. All rights reserved.

	• User identity management		
EventService	•Create/edit/submit/approve events • Publish & search • Notify on state changes • Cache hot read	IEventsAPI	IPolicyCheck IMongoDB IRedisCache INotificationAPI
RegistrationService	• Register/cancel • Waitlist management • QR check-in • Seat locks	IRegistrationAPI	IPolicyCheck IMongoDB IRedisLocks IWaitlistQueue
PaymentService	• Create checkout sessions • Process refunds • Verify webhooks • Audit & notify	IPaymentsAPI	IPaymentGateway IMongoDB IAuditLog INotificationAPI
NotificationService	• Templated emails/SMS • Confirmations & reminders • Feedback prompts • Failure logging	INotificationAPI	IEmailProvider IAuditLog
CalendarService	• ICS feed generation • Add-to-calendar actions • Calendar integrations	ICalendarAPI	ICalendarProviders
AnalyticsService	• Attendance/revenue aggregations • Feedback analysis • CSV exports • Role-aware access	IReportsAPI	IMongoDB IPolicyCheck

Table 11: Domain Service Layer

- **Async/Background Layer**

Component	Responsibilities	Provided Interfaces	Required Interfaces

Copyright © 2025 Tony D Barber. All rights reserved.

Waitlist Queue	• Event stream for waitlist operations • Queue waitlist.enqueue events • Queue promotion.requested events	IWaitlistQueue	None
Registration Worker	• Consume queue messages • Execute promoteWaitlist() • Send promotion notifications • Batch processing	None	IWaitlistQueue IMongoDB IRedisLocks INotificationAPI

Table 12: Async Background Layer

- **Infrastructure Layer**

Component	Responsibilities	Provided Interfaces	Required Interfaces
MongoDB	• Primary persistence • Collections: users, events, registrations, payments, feedback, audit logs • CRUD & aggregations	IMongoDB	None
Redis	• Seat locks • Hot caches • Rate limits • Keys: event:{id}:seats, lock:event:{id}	IRedisCache IRedisLocks	None
Audit Logger	• Append-only audit trail • Log sensitive actions • Write to audit collection	IAuditLog	IMongoDB

Copyright © 2025 Tony D Barber. All rights reserved.

Table 13: Infrastructure Layer

- **External Systems**

Component	Responsibilities	Provided Interfaces	Required Interfaces
CAS SSO	• University SSO authentication • Ticket validation • User identity verification	ICAS	None
Payment Gateway	• Hosted checkout • Process payments & refunds • Payment webhooks	IPaymentGateway	None
Email/SMS Provider	• Outbound email delivery • SMS notifications • Template rendering	IEmailProvider	None
Calendar Providers	• Calendar ingestion • Add to Google/Outlook/iCal • ICS consumers	ICalendarProviders	None

Table 14: External Systems

Copyright © 2025 Tony D Barber. All rights reserved.

4.3 Deployment Diagram

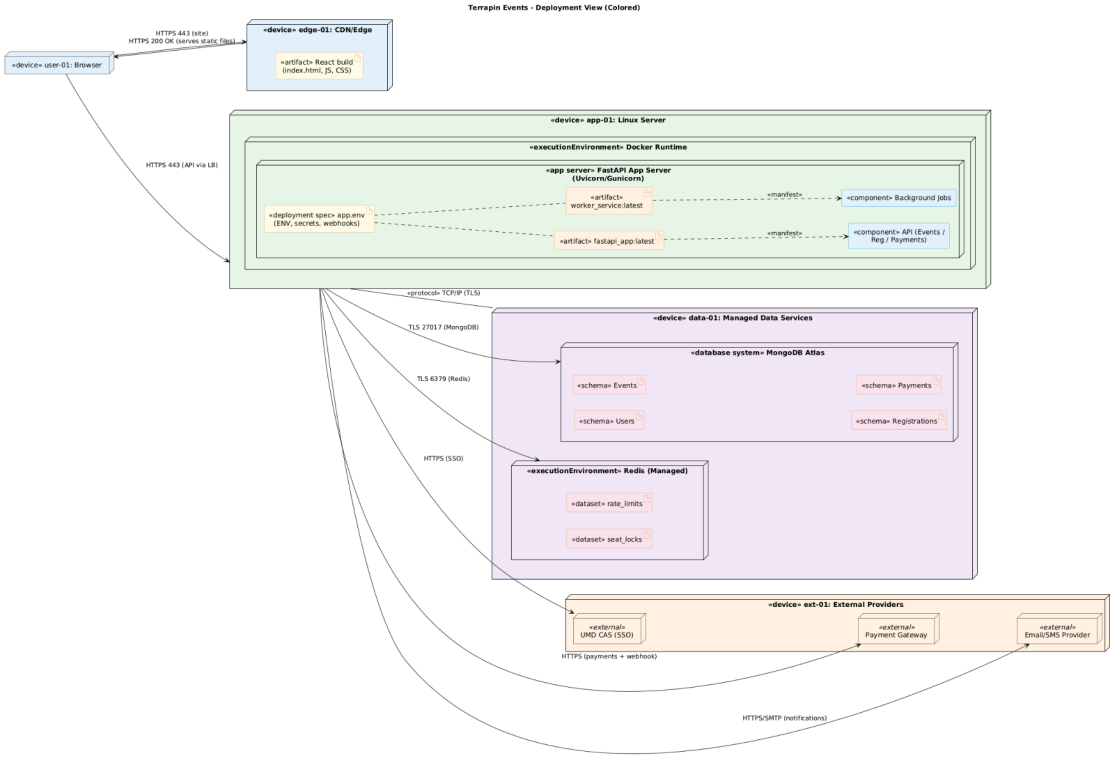


Figure 15: UML Deployment Diagram

Diagram: ***UML Deployment Diagram***

4.3.1 Description:

The deployment diagram illustrates the physical deployment architecture of the Terrapin Events system across user devices, servers, managed data services, and third-party providers. It highlights how the application executes in real environments, the network boundaries, and the runtime interactions between components.

1. User Interaction Layer

- End-users access the system through the browser (device user-01) using HTTPS over port 443.
- Static frontend assets such as React build files (index.html, JS, CSS) are served from a CDN/Edge node (device edge-01) to reduce latency and improve global performance.
- The browser communicates with the backend APIs exclusively via secure HTTPS requests routed through a load balancer.

2. Application Server Layer

Copyright © 2025 Tony D Barber. All rights reserved.

- The application backend runs on device app-01, a Linux server hosting a Dockerized runtime environment.
- Two main FastAPI containers run here:
    - fastapi_app:latest—exposes core API endpoints (Events, Registrations, Payments, Users).
    - worker_service:latest—handles background jobs such as waitlist promotion, notifications, and scheduled tasks.
- The environment configuration (secrets, ENV, webhook URLs) is injected via a separate deployment specification module (app_env).
- The app server communicates directly with managed databases and external providers using TLS-encrypted connections.

3. Managed Data Services Layer

a. MongoDB Atlas (database system)

- Stores all persistent application data across different schemas:
    - Events
    - Users
    - Payments
    - Registrations
- Connectivity to MongoDB occurs over TLS 27017, ensuring secure encrypted data transfer.

b. Redis (execution environment – managed)

- Used for:
    - Seat locking (preventing oversubscription during registrations)
    - Rate limiting (preventing abuse, BOT activity, DDoS attempts)
    - Caching frequently accessed values
- Communication uses TLS 6379 for secured in-transit encryption.

4. External Provider Layer

This layer includes systems outside the Terrapin Events environment:

a. UMD CAS SSO

- Handles secure university login and identity verification.
- The application sends ticket validation requests over HTTPS for authentication.

b. Payment Gateway

- Processes online payments for paid events.
- Communication includes real-time payment session creation and asynchronous webhook callbacks.

c. Email/SMS Provider

Copyright © 2025 Tony D Barber. All rights reserved.

- Sends confirmations, waitlist promotions, reminders, and payment receipts.
- Interacts with backend via HTTPS/SMTP channels.

5. Connectivity Overview

- All critical paths (API traffic, DB queries, Redis connections, CAS login, payment workflows) use TLS-encrypted communication, ensuring compliance with privacy and security standards.
- The architecture is fully decoupled: frontend, backend, database, and background workers operate as independent deployable units.
- External providers integrate seamlessly through well-defined HTTP(S) interfaces.

4.3.2 Summary:

The deployment diagram shows the physical runtime architecture of Terrapin Events, detailing how the React frontend, Dockerized FastAPI services, managed databases (MongoDB, Redis), and external providers (CAS SSO, Payment Gateway, Notification Service) interact securely over HTTPS/TLS within a distributed cloud environment.

4.4 Behavioral Diagrams

4.4.1 Relationships among components depicted via Sequence Diagrams

Scenario 1: Login via CAS SSO (Authentication & RBAC)

- The user is redirected to UMD CAS for authentication, ensuring secure SSO login.
- AuthService validates the CAS ticket, issues a JWT, and retrieves user roles.
- The user is directed to their dashboard (Student/Organizer/Admin) based on RBAC.

Copyright © 2025 Tony D Barber. All rights reserved.

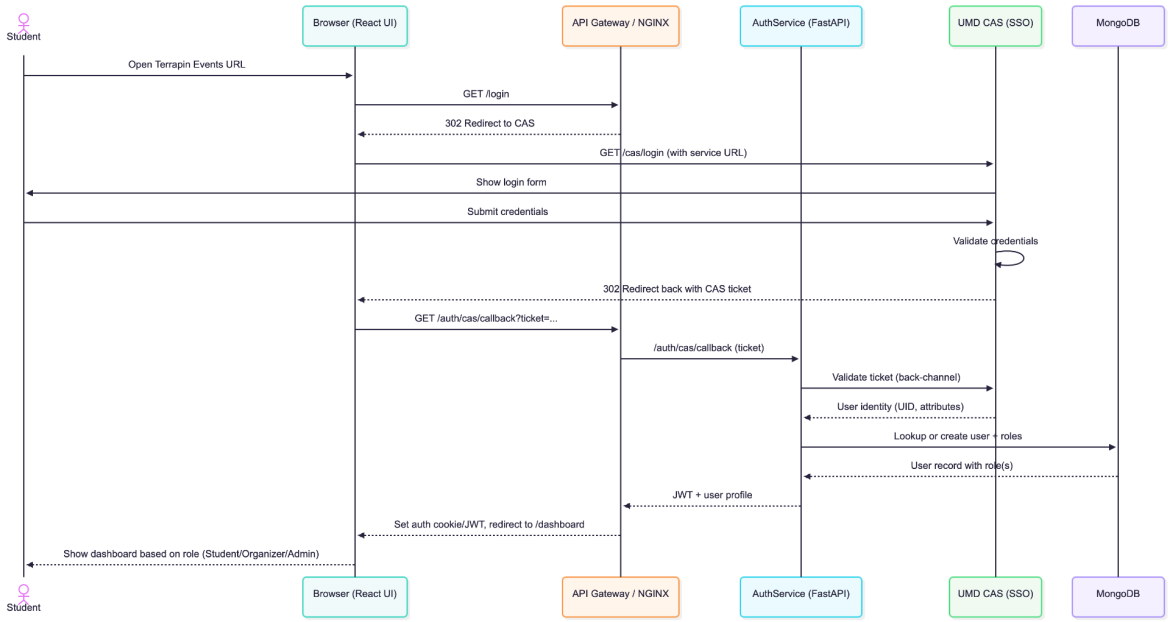


Figure 16: Sequence Diagram for CAS Authentication and Role-Based Access Control

Diagram: ***Scenario 1***

Scenario 2: Student Registers for Approved Free Event

- A student initiates registration; the system validates event status and availability.
- Redis lock ensures atomic seat allocation, preventing oversubscription during high load.
- Registration is stored in the database, and a confirmation is sent via Notification Service.

Copyright © 2025 Tony D Barber. All rights reserved.

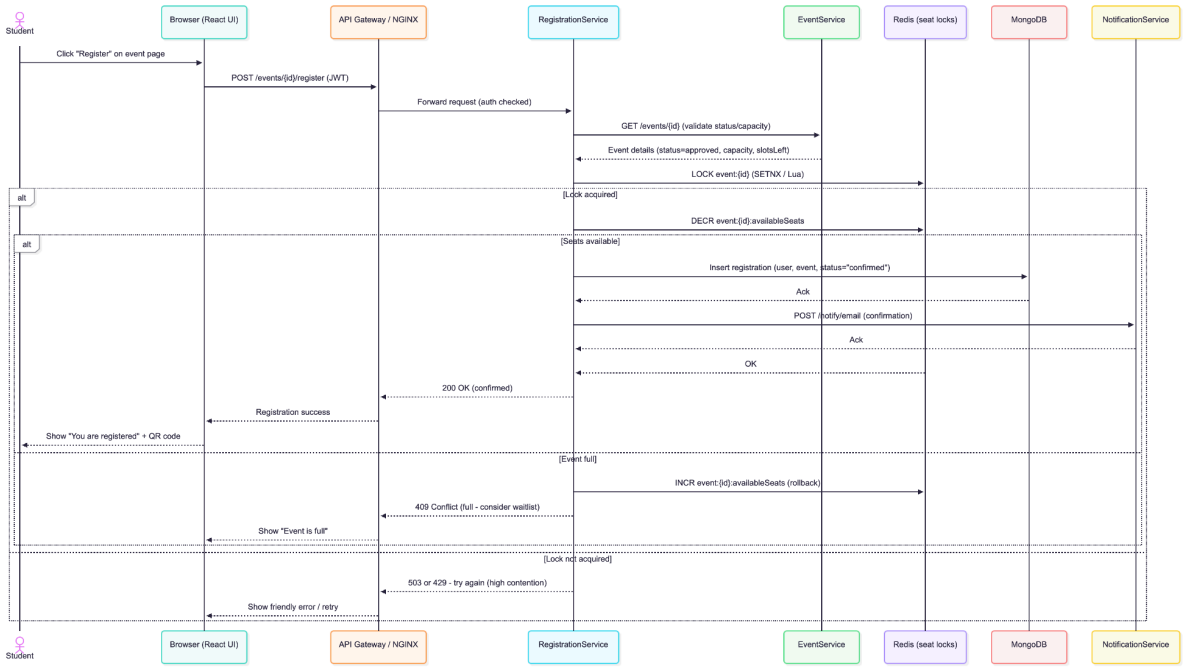


Figure 17: Sequence Diagram for Free Event Registration with Distributed Seat Locking

Diagram: ***Scenario 2***

Scenario 3: Student Registers for Paid Event

- Registration is marked *pending* while PaymentService creates a checkout session.
- Payment Gateway confirms the transaction via webhook, ensuring tamper-proof results.
- Registration is updated to *confirmed*, and the user receives a payment receipt.

Copyright © 2025 Tony D Barber. All rights reserved.

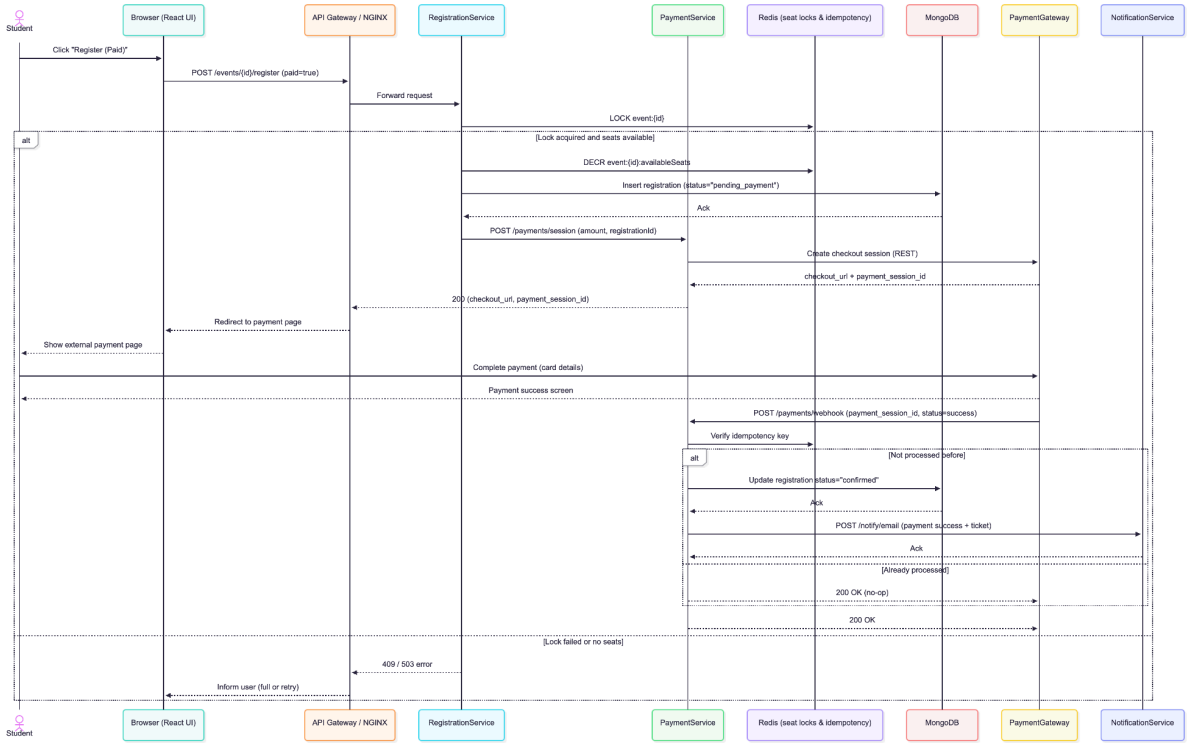


Figure 18: Sequence Diagram for Paid Event Registration with External Payment Webhook Integration

Diagram: ***Scenario 3***

Scenario 4: Waitlist Promotion When Seat Opens (Async Worker)

- Cancelled registrations increase seat count; an async worker polls waitlist events.
- Worker promotes the next eligible waitlisted user and sends a notification.
- Atomic updates ensure accurate seat accounting and avoid race conditions.

Copyright © 2025 Tony D Barber. All rights reserved.

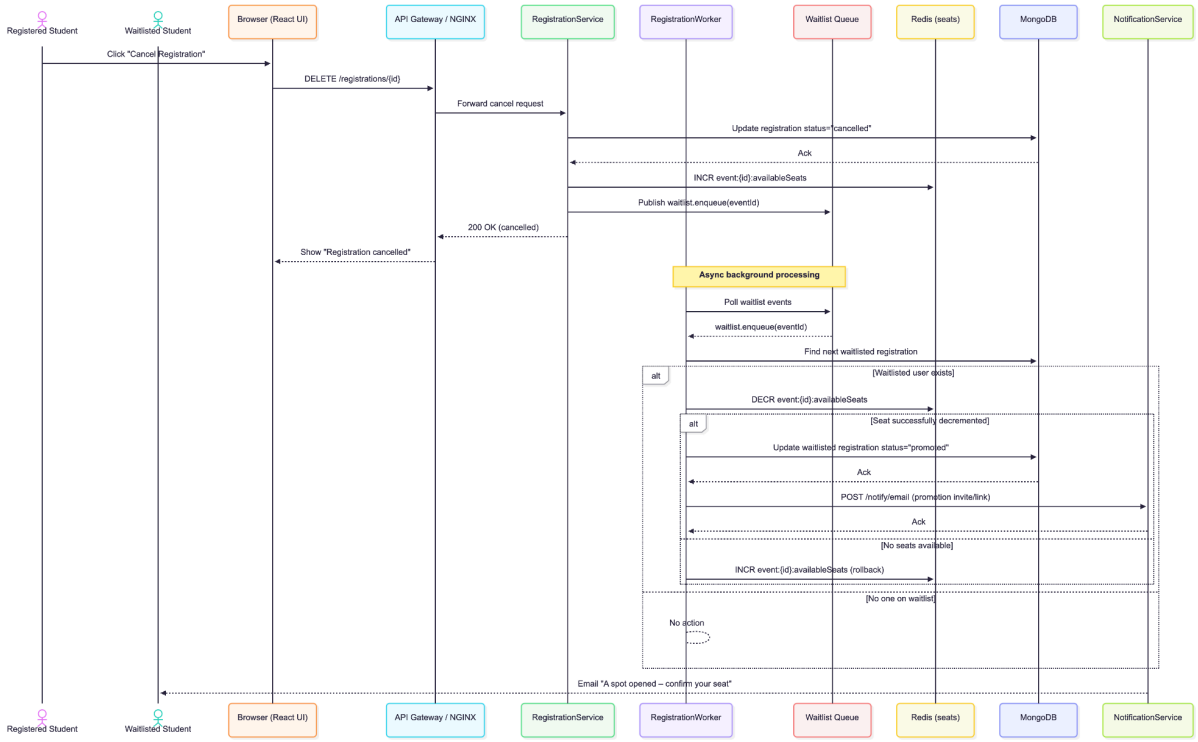


Figure 19: Sequence Diagram for Waitlist Promotion via Asynchronous Background Worker

Diagram: *Scenario 4*

Scenario 5: Unauthorized Role Escalation Attempt Blocked (Abuse Case AC-01)

- Malicious user attempts to escalate privileges by modifying the API payload.
- AuthService applies RBAC checks and denies the request, logging it for audit review.
- AuditService records the suspicious activity for compliance and security monitoring.

Copyright © 2025 Tony D Barber. All rights reserved.

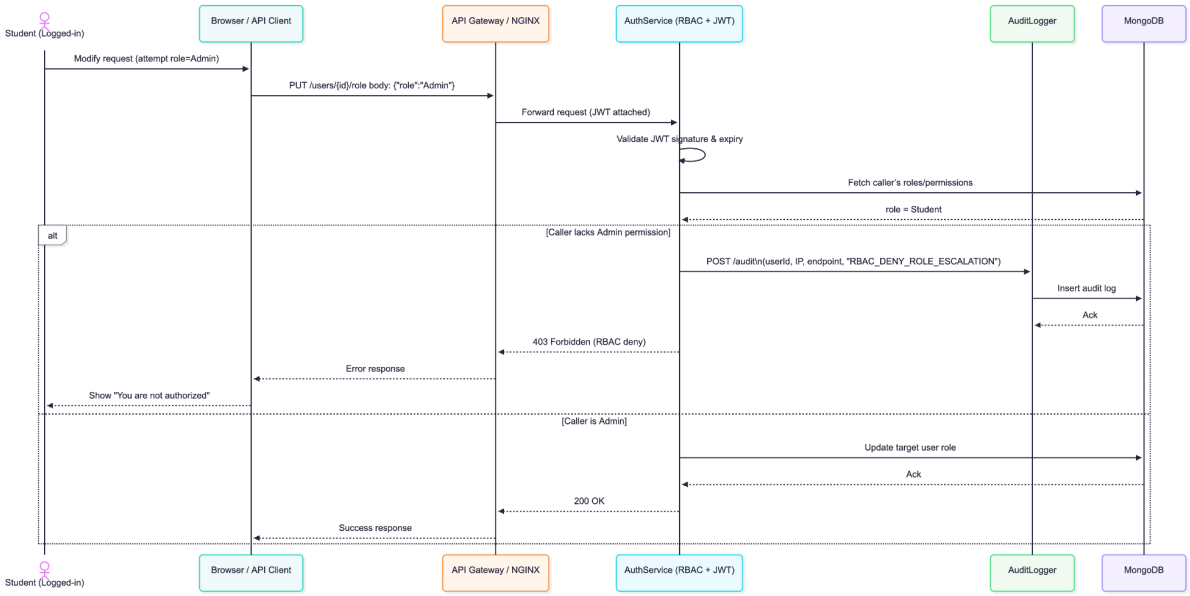


Figure 20: Sequence Diagram for RBAC Enforcement During Unauthorized Role Escalation Attempt

Diagram: ***Scenario 5***

Copyright © 2025 Tony D Barber. All rights reserved.

5 Architectural Decisions and Evaluation

5.1 Key Design Decisions and Rationale

The following are the key design decisions made and the rationale behind them.

5.1.1 Layered Client-Server Architecture on Kubernetes with NGINX Ingress

Deploy Terrapin Events as a layered client-server system inside a Kubernetes cluster, with NGINX Ingress as the single external entry point, routing traffic to independent Frontend, API, and Assets pods. MongoDB Atlas is hosted as a managed database outside the cluster.

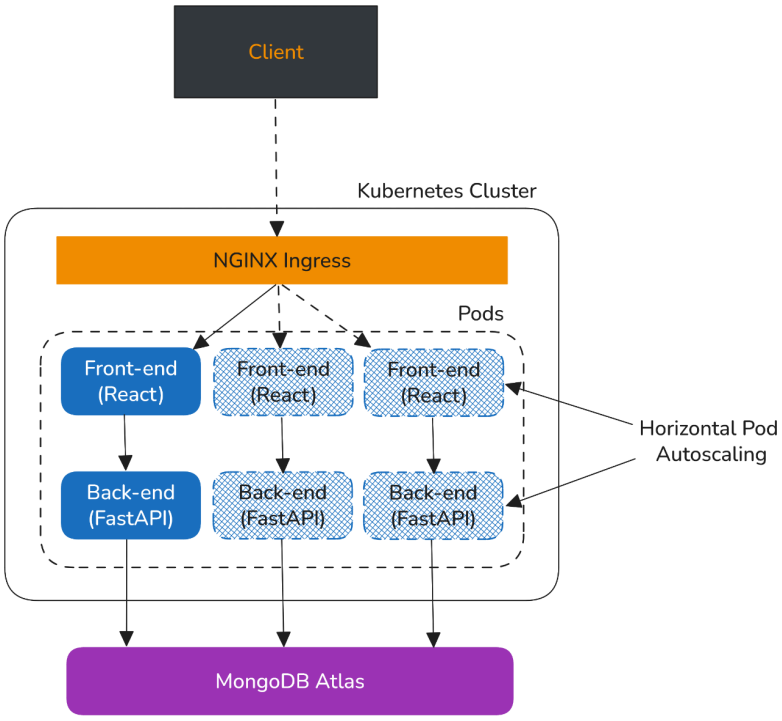


Figure 21: Layered Client-Server Architecture on Kubernetes with NGINX Ingress

Rational and Trade-offs

- ➔ **Kubernetes** horizontal pod autoscaling directly addresses **Scalability** and **Availability** requirements (handle thousands of concurrent users, maintain ≥ 99% uptime, and maintain p95 latency targets for search/registration).
- ➔ **NGINX Ingress** centralizes **TLS** termination, rate limiting, and routing rules, simplifying security controls and supporting abuse-case mitigations such as registration floods.
- ➔ Using **managed MongoDB Atlas** offloads backup, replication, and maintenance responsibilities and supports the NFRs for **Data Integrity & Backup** and **Reliability**.
- ➔ Aligns with the system's **3-tier model** (presentation, application, data layers) defined in the Architectural Design section above and SRS

Copyright © 2025 Tony D Barber. All rights reserved.

This decision directly supports the following requirements and quality attributes:

- ➔ **FR001** Authentication & Access Control, **FR003** Registration & Waitlist, **FR006** Attendance Tracking & Reporting from the Project Management Plan. Cluster ensures these critical flows remain responsive under load.
- ➔ **NFR001** Reliability & Availability, **NFR002** Security & Compliance, **NFR003** Scalability, **NFR007** Data Integrity & Backup.

5.1.2 MVC Pattern Across Frontend and Backend

We adopt the **Model-View-Controller (MVC)** architectural pattern to enforce a clean **separation of concerns** across both the frontend and backend. On the client side, the frontend is implemented using **React**, where views handle user interaction while a dedicated service layer communicates with the API, and application state is maintained through **client-side models**. On the server side, the backend is structured with **FastAPI** controllers that expose well-defined endpoints, each delegating business logic to a service layer that manages the underlying data models stored in **MongoDB**. This unified MVC approach ensures consistency, modularity, and maintainability across the entire application stack.

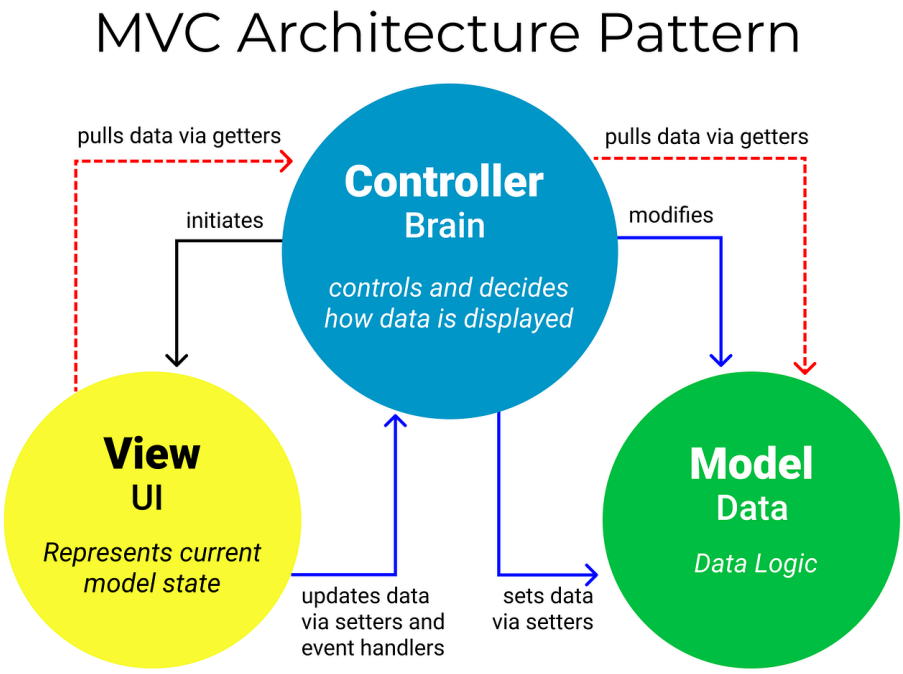


Figure 22: MVC Pattern

Rational and Trade-offs

- ➔ Clean separation of concerns makes it easier to evolve UI, business rules, or persistence independently, directly supporting Maintainability and Modifiability targets (**NFR008**)
- ➔ MVC, combined with **role-based views** (Student, Organizer, Admin), supports usability and accessibility requirements for different user classes described in the SRS

Copyright © 2025 Tony D Barber. All rights reserved.

- ➔ Slight overhead in boilerplate (Data Transfers, Data mapping to models) is accepted in exchange for testability and clarity

This decision directly supports the following requirements and quality attributes:

- ➔ **FR002** Event Creation & Approval, **FR007** Search & Discovery, **FR009** Calendar Integration, isolated controllers and views for each capability.
- ➔ **NFR005** Usability & Accessibility, **NFR006** Maintainability.

5.1.3 CAS and Role-Based Access Control for Identity and Access Management

Integrate **UMD CAS SSO** as the sole authentication mechanism and implement role-based access control for Students/Faculty and Organizers.

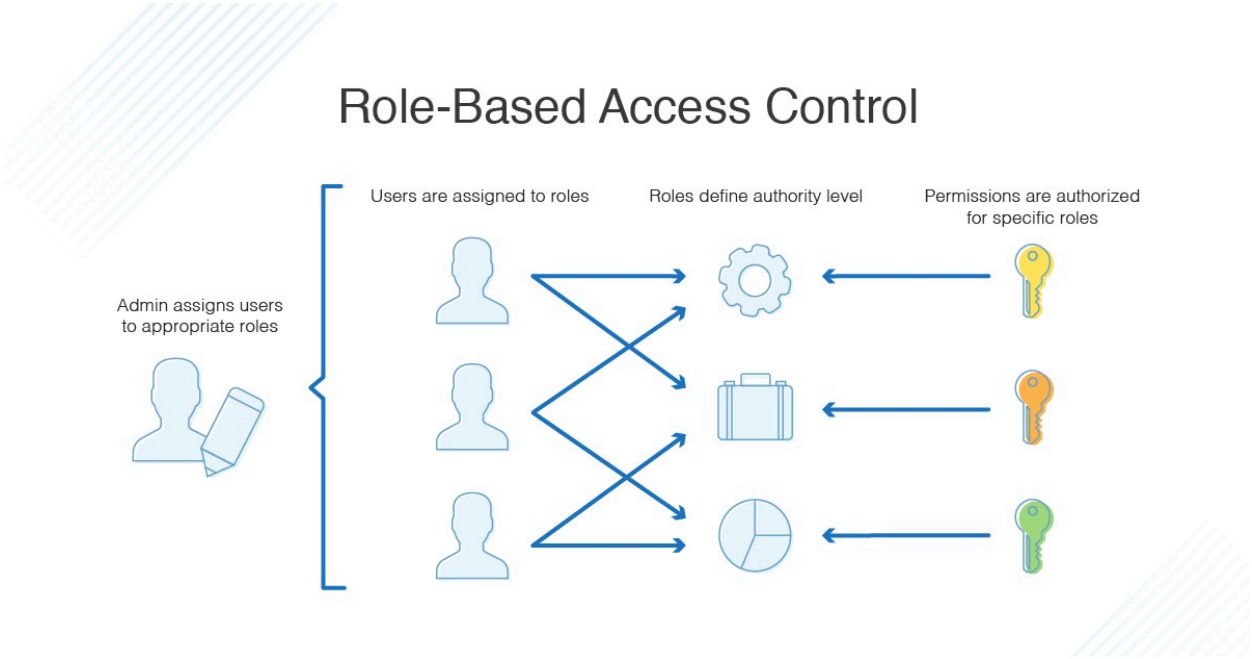


Figure 23: RBAC Access Control

Rational and Trade-offs

- ➔ Satisfies **university policy** and aligns with **FERPA** and institutional compliance.
- ➔ Centralized identity management reduces credential sprawl and enables uniform enforcement of abuse-case protections (**AC-01** Unauthorized Role Escalation, **AC-02** Event Data Manipulation).
- ➔ Adds dependency on CAS availability (**AR-01** from the Risk and Mitigation section), mitigated through health checks, retries, and user-friendly fallbacks.

This decision directly supports the following requirements and quality attributes:

- ➔ **FR001** Authentication & Access Control
- ➔ Abuse case defense for **AC-01**, **AC-02, and AC-06**.

Copyright © 2025 Tony D Barber. All rights reserved.

5.1.4 Secure Payment Gateway Integration with Webhooks and No Card Storage

Use a university-approved external payment gateway for all paid events. The backend handles checkout session creation and webhook-based confirmation. No card data is stored in Terrapin Event Servers.

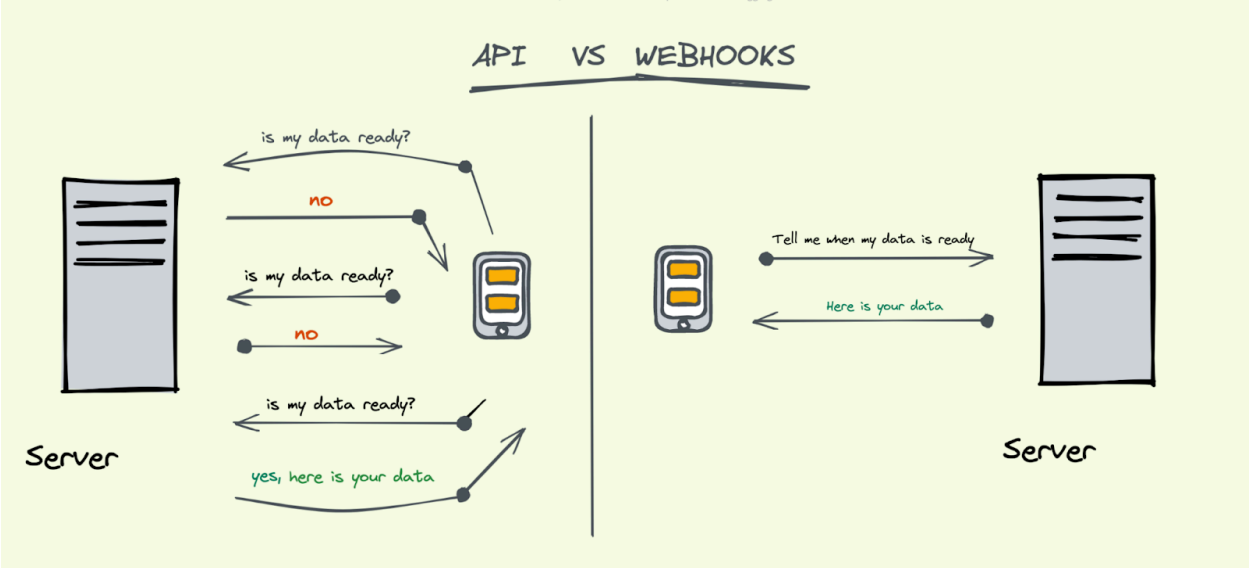


Figure 24: API vs Webhooks

Rational and Trade-offs

- ➔ Offloads **PCI DSS** (Payment Card Industry Data Security Standard) responsibilities to the gateway while still supporting event registration and refunds.
- ➔ Webhook-driven confirmation introduces eventual consistency (webhooks for updates will not have data consistent across all parts of the system immediately). This is mitigated with idempotent handlers, queues, and audit logs (risks **AR-06**, **AR-09**)

This decision directly supports the following requirements and quality attributes:

- ➔ **FR004** Payment Gateway Integration, **FR006** Reporting of financial transactions.
- ➔ Security & Compliance, Data Integrity, Reliability.

5.1.5 Containerization And CI/CD Approach

Package all components as Docker containers, deploy via Kubernetes (Akamai Connected Cloud), and enforce CI/CD pipelines with automated tests and health-checked rollouts.

Rational and Trade-offs

- ➔ **NFR-013** requires containerized deployment and automated promotion from staging to production.
- ➔ This design minimizes configuration drift and supports rolling updates.
- ➔ However, this approach demands initial setup effort and planning.

This decision directly supports the following requirements and quality attributes:

- ➔ Maintainability, Deployability, Availability, and Operational Risk mitigation (**AR-08 to AR-11**)

Copyright © 2025 Tony D Barber. All rights reserved.

5.2 Architectural Tradeoffs and Alternatives

5.2.1 Architectural Tradeoffs of the Selected Layered Client-Server Architecture

The finalized architecture is a **layered client-server model** deployed on **Kubernetes** with **NGINX Ingress**. It employs a **React-based** frontend, **FastAPI** backend services, and **MongoDB Atlas and** provides a scalable, secure, and maintainable foundation that aligns with the project's **SRS** and **quality goals**. However, this architecture also introduces several tradeoffs that must be acknowledged.

The separation of concerns across layers enables targeted scaling and clear modularity, directly supporting performance, availability, and maintainability objectives. At the same time, it increases operational overhead and requires strong coordination across components, deployment pipelines, and infrastructure.

The following are some of the key tradeoffs:

➔ **Operational Complexity**

Kubernetes, containerization, ingress rules, autoscaling policies, and secret management introduce overhead beyond what a monolithic deployment would require.

➔ **Increased Infrastructure Cost**

Maintaining a Kubernetes cluster, managed database, and caching layers provides long-term scalability but has a higher baseline cost than a single-server or monolithic deployment approach.

➔ **Higher Coordination Effort**

Implementing new features may require synchronized updates across multiple layers (React view, API controller, service logic, database model), increasing implementation time for small modifications.

➔ **Eventual Consistency in Asynchronous Workflows**

Queue-based background tasks (payments, emails, waitlist) improve performance but result in non-immediate visibility of results, which requires monitoring and acceptance from stakeholders.

Justification For Selecting Kubernetes Based Layered Client-Server Architecture:

➔ **Meets prioritized NFRs**
- Horizontal pod autoscaling for the API and frontend aligns with scalability and performance scenarios.
- Managed MongoDB Atlas and containerized services support reliability, backup, and recovery objectives (RPO/RTO), while NGINX Ingress plus TLS and RBAC fulfill security and compliance needs

➔ **Balanced complexity**
- More modular and resilient than a monolith, but less operationally complex than a fine grained serverless or microservices landscape.
- Fits the team's skills and the semester timeline while still demonstrating modern industry practices (containers, Kubernetes, autoscaling).

➔ F**uture evolution path**

Copyright © 2025 Tony D Barber. All rights reserved.

- The current decomposition into frontend, API, and assets services on Kubernetes provides a clear path to evolve into more fine grained services later (e.g., splitting registration, payments, notifications) without an entire architecture rework.

5.2.2 Alternative Designs that were considered initially

Monolithic Architecture

This was considered for its Familiarity to many developers, straightforward deployment, easier to debug in simple environments. It provisions a single deployable unit combining UI, business logic, and persistence into one application and database instance.

Reasons For Rejection:

➔ **Inefficient Scaling**

Scaling the system for peak events would mean scaling the entire monolith even if only registration endpoints are under load, conflicting with Scalability and Cost goals.

➔ **Single Point of Failure**

Any defect in one part of the monolith could take down the whole platform, jeopardizing Availability ≥ 99% during critical registration windows.

➔ **Deployment Risk**

Even a small change would require redeploying the entire application, increasing downtime risk and violating deployment NFRs targets.

➔ **Team Productivity**

Given the multi-role team and sprint based plan, a monolith would create merge pressure and slower parallel development compared to a layered, modular system.

Justification:

A monolith would slightly reduce short-term infrastructure and operational complexity, but at significant cost to scalability, availability, and modifiability. Qualities that are explicitly prioritized by stakeholders. Therefore, it was rejected in favor of a layered, containerized deployment on Kubernetes.

Serverless Architecture

This was considered for its ability to decompose the backend into fine-grained functions (e.g., registration, notifications, payments) deployed to a function as a service platform.

Reasons For Rejection:

➔ **Limited Control and Customization**

Deep integration with CAS, payment gateways, and campus networking policies requires control over runtime, networking, and observability that typical serverless platforms restrict.

➔ **Cost Profile**

Copyright © 2025 Tony D Barber. All rights reserved.

Terrapin Events experiences bursts (registration surges) but also sustained daily usage, per request pricing and cold start overhead could increase operational costs relative to right sized Kubernetes nodes, conflicting with project budget constraints.

➔ **Complexity of Stateful Workflows**

Event registrations, waitlists, and idempotent payment flows benefit from long-lived services and Redis coordination rather than distributed stateless functions.

Justification:

Serverless offers attractive autoscaling and reduced cluster management, but it introduces cold start latency, less control over security posture, higher integration complexity, and potential vendor lock in. This conflicts with top-ranked quality attributes (security, performance, availability) and deployment constraints. The team therefore rejected a fully serverless architecture in favor of Kubernetes-based containers with horizontal pod autoscaling.

5.3 Scenario-Based Architectural Evaluation

Architectural evaluation was performed primarily through **scenario-based analysis**, leveraging the Quality Attribute Scenarios and abuse cases defined in the SRS and mapping them onto the Kubernetes plus MVC architecture. Below are representative scenarios and conclusions.

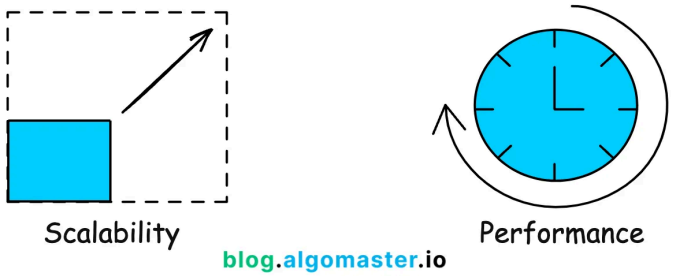


Figure 25: Peak Registration Load

Scenario 01 (SC-01) - Peak Registration Load (Performance & Scalability)			
Stimulus	**Environment**	**Response and Architectural Support**	**Result**
1,000+ concurrent students attempt to register for high-demand events during a 15 minute window.	Production cluster, HPA configured for API pods, Redis cache and locks enabled.	NGINX Ingress distributes incoming traffic across multiple API pod replicas Kubernetes horizontally scales pods based on	Scenario walk-through shows that with autoscaling and Redis backed locking, the architecture can meet p95 latency goals and maintain data

Copyright © 2025 Tony D Barber. All rights reserved.

Scenario 01 (SC-01) - Peak Registration Load (Performance & Scalability)			
		CPU/latency. Registration endpoints use Redis for seat counters and idempotency keys, preventing over-booking and duplicate registrations Long-running tasks (emails, waitlist promotion) are offloaded to queues, keeping synchronous registration latency within target bounds	consistency, addressing NFR-001 and QAC-006.

Table 15: Scenario 1 - Peak Registration Load

Scenario 02 (SC-02) - Unauthorized Role Escalation (Security: AC-01)			
Stimulus	**Environment**	**Response and Architectural Support**	**Result**
A logged-in student attempts to modify their role to "Administrator" via direct API ma\nipulation.	Normal operations, CAS SSO and RBAC active.	NGINX Ingress terminates TLS and forwards to API. FastAPI middleware validates JWT and roles issued after CAS login. RBAC policy layer denies any mutation to role assignments unless request originates from Admin context. Audit logging component records the attempt with user ID, IP, timestamp.	Scenario demonstrates that the architecture blocks privilege escalation attempts, preserves integrity of role data, and produces an auditable trail, satisfying abuse case acceptance criteria (100% of unauthorized attempts blocked) and reinforcing the Security quality attribute.

Copyright © 2025 Tony D Barber. All rights reserved.

Table 16: Scenario 2 - Unauthorised Role Escalation

Scenario 03 (SC-03) - Node Failure During Event Registration (Availability)			
Stimulus	**Environment**	**Response and Architectural Support**	**Result**
A Kubernetes worker node hosting one API pod fails mid-registration.	Multi-node cluster, replicas of API and Frontend pods scheduled across nodes, MongoDB Atlas replica set.	Kubernetes automatically reschedules affected pods on healthy nodes, readiness/liveness probes and NGINX Ingress ensure traffic is only routed to healthy pods. MongoDB Atlas maintains availability through replica sets, clients automatically reconnect. In-flight registrations rely on idempotency keys and Redis locks so that retries complete safely without duplicate rows.	Scenario analysis shows that a single node failure degrades capacity but not correctness. The system continues to meet Availability goals (≥99% uptime, bounded error rate) with graceful degradation.

Table 17: Scenario 3 - Node Failure During Event Registration

Scenario 04 (SC-04) - Safe Rollout of New Feature (Maintainability & Deployability)			
Stimulus	**Environment**	**Response and Architectural Support**	**Result**
The team deploys a new endpoint/feature during the semester.	CI/CD pipeline with automated tests, container images, and Kubernetes rolling updates.	Changes are localized to a backend module and associated React view due to the layered MVC structure. CI pipeline executes unit and integration	Scenario validates that the architecture supports rapid, low-risk deployments, aligned with maintainability and deployability NFRs and project management

Copyright © 2025 Tony D Barber. All rights reserved.

Scenario 04 (SC-04) - Safe Rollout of New Feature (Maintainability & Deployability)			
		tests, on success, a new Docker image is deployed with health checked rolling updates. Automatic rollback occurs on failed health checks.	expectations for iterative delivery.

Table 18: Scenario 4 - Safe Rollout of New Feature

Summary:

With the evaluation above we can confidently conclude that the Kubernetes based layered client-server architecture with MVC, CAS SSO, managed MongoDB, and asynchronous workers demonstrates strong alignment with the prioritized quality goals from the SRS and the project's objectives in the PMP. It allies for a secure, performant, highly available, user-friendly, and maintainable event management for the UMD campus.

Copyright © 2025 Tony D Barber. All rights reserved.

6 Traceability

6.1 SRS-to-Architecture Trace Matrix

Diagram: *Traceability Matrix*

SRS-to-Architecture Traceability Matrix

Functional Requirements → Architecture Mapping

FR ID	Requirement	Architecture Component(s)
FR-1	Authenticate all users via UMD CAS	API Gateway, AuthService (CAS integration), JWT Issuer
FR-2	Initiate user session after login	AuthService, API Gateway session middleware
FR-3	RBAC for all roles	AuthService (RBAC), API Gateway policy checks
FR-4	View user profiles	ProfileService, MongoDB
FR-5	Update profile	ProfileService, MongoDB
FR-6	Create & manage events	EventService, MongoDB
FR-7	Edit draft events	EventService, MongoDB version store
FR-8	Version history	EventService versioning subsystem
FR-9/10	Admin review & approval	EventService workflow engine, AuthService (Admin role)

Figure 26: Screenshot of SRS-to-Architecture Traceability Matrix

6.2 Quality Attribute Scenarios Traced to Architecture Tactics

1. Performance

Scenario ID	Scenario Summary	Architectural Elements / Tactics
P1- Search latency	Student submits keyword search; p95 ≤ 2s, p99 ≤ 3s.	React Frontend issues lightweight queries → NGINX Ingress load-balances → API uses indexed MongoDB Atlas queries, horizontal scaling meets NFR-001 & NFR-002.

Copyright © 2025 Tony D Barber. All rights reserved.

P2 - Registration confirmation	Attendee clicks “Register”; confirmation ≤ 2s; email ≤ 5s.	Ingress → API routes; MongoDB Atlas stores registration API triggers email notifications; stateless pods scale automatically.
P3 - Checkout (paid events)	Payment completes ≤ 4s; ≥ 99.5% success.	API integrates with Payment Gateway using tokenization MongoDB stores transaction status; TLS + cloud hosting ensures low-latency secure requests.

Table 19: Performance

2. Availability & Reliability

Scenario ID	Scenario Summary	Architectural Elements / Tactics
A1 - Uptime SLO	Service must maintain ≥ 99% monthly uptime.	Kubernetes on Akamai Cloud with multi-pod Frontend/API/Assets replicas, health probes, and automatic restarts, MongoDB Atlas HA replicas.
A2 - External gateway outage	Payment provider returns 5xx, system must fail gracefully.	API’s payment adapter handles retries and error states MongoDB order remains pending; user sees clear message; no duplicate charges (AC-07).
A3 - Backup & restore	Restore last good state; RPO ≤ 24h, RTO ≤ 4–8h.	MongoDB Atlas automated backups + cloud snapshots PM plan includes DR steps and accidental data loss mitigation.

Table 20: Availability and Reliability

3. Security & Compliance

Scenario ID	Scenario Summary	Architectural Elements / Tactics
S1 - RBAC enforcement	Unauthorized organizer attempts edit; must be denied.	CAS SSO for authentication, API enforces RBAC (FR-003), ownership checks, audit logs recorded (FR-020, AC-01/AC-02).
S2 - Sensitive export guard	Export of PII requires elevated role + masking.	Admin dashboard gated by role; API checks privilege + justification, all exports logged with user attribution (FR-019, AC-06).
S3 - Card data handling	The system must not store card data.	Payment Gateway tokenization, checkout UI uses hosted fields, MongoDB stores only payment status, PCI DSS compliance (NFR-006).

Copyright © 2025 Tony D Barber. All rights reserved.

Table 21: Security and Compliance

4. Usability & Accessibility

Scenario ID	Scenario Summary	Architectural Elements / Tactics
U1 - "Find & register" task	Students should finish in 3–5 clicks & ≤ 90s.	React SPA with optimized routing; event catalog, filters, and clean UI flows, Ingress/API interactions hidden from user matches NFR-010.
U2 - Accessibility conformance	WCAG 2.1 AA: screen readers & keyboard navigation.	Semantic HTML, ARIA roles, proper focus order, accessible components, color-contrast rules, compliant with NFR-009.

Table 22: Usability and Accessibility

5. Scalability

Scenario ID	Scenario Summary	Architectural Elements / Tactics
C1 - Flash registration window	5,000 concurrent students, minimal degradation.	Kubernetes horizontal pod autoscaling NGINX Ingress load balances traffic MongoDB Atlas scales reads/writes NFR-002 supports 2× load.

Table 23: Scalability

6. Maintainability & Deployability

Scenario ID	Scenario Summary	Architectural Elements / Tactics
M1 - MTTR Sev-2	Fix & deploy patch within ≤ 8h.	Git-based CI/CD pipeline Docker containers for API/Frontend structured logs & metrics (NFR-012) simplify diagnosis, rolling updates.
M2 - Safe dependency upgrades	Canary + rollback for library updates.	Modular containerized services CI pipeline verifies builds automatic rollback supported by metrics & error thresholds.

Table 24: Maintainability and Deployability

7. Data Integrity

Copyright © 2025 Tony D Barber. All rights reserved.

Scenario ID	Scenario Summary	Architectural Elements / Tactics
D1 - Waitlist auto-promotion	FIFO promotion with full audit trail.	API enforces FIFO logic and atomic updates to MongoDB audit logs capture each movement (FR-012 + FR-020).
D2 - Duplicate registration prevention	Double-clicking "Register" must create only one entry.	API uses idempotency keys & unique (user, event) constraints in MongoDB prevents duplicates (AC-03).

Table 25: Data Integrity

6.3 Bidirectional linkage (requirements ↔ architecture)

The Terrapin Events architecture maintains full bidirectional traceability between the SRS and the system's structural design. Each functional requirement (FR), non-functional requirement (NFR), and abuse case (AC) is linked to one or more architectural components, and every architectural component is justified by at least one requirement in the SRS. This ensures that the architecture is both requirements-driven and verifiable.

Forward Traceability (SRS → Architecture)

Forward traceability is demonstrated in the SRS-to-Architecture Traceability Matrix
Examples include:

- **FR-1: CAS Authentication** → AuthService, API Gateway, JWT Issuer
- **FR-13: Event Registration** → RegistrationService, Redis seat-lock, MongoDB
- **FR-17: Payments** → PaymentService, External Gateway, Audit Logging
- **NFR-1: Performance** → Redis caching, async FastAPI, CDN edge caching
- **NFR-4: Security** → CAS SSO, RBAC, TLS 1.3, signed tokens
- **AC-1/AC-2 (Stolen credentials, privilege escalation)** → RBAC engine, CAS verification, write-once audit logs

This shows how every requirement is realized through one or more architectural structures, tactics, or deployment decisions.

Backward Traceability (Architecture → SRS)

Each architectural element is included because it satisfies one or more SRS requirements:

- **AuthService** exists specifically to enforce **FR-1, FR-3**, and abuse-case protections **AC-1, AC-2**.
- **EventService** is justified by **FR-6–FR-12** (event creation, editing, publishing, versioning, search).
- **RegistrationService + Redis** exist because of **FR-13–FR-15** and the performance NFRs.
- **PaymentService** is required by **FR-17–FR-18** and NFR-6 (PCI compliance).
- **NotificationService** directly maps to **FR-19** and AC-8 (anti-phishing controls).
- **MongoDB Atlas + backups** meet **NFR-3, NFR-7** (availability, recovery).
- **API Gateway / NGINX Ingress** enforces **NFR-4**, AC-10 (DoS protection), and routing rules from the SRS.
- **CI/CD + Containerization** map directly to **NFR-13 and NFR-14**.

End-to-End Lifecycle Alignment

Together, these matrices establish complete bidirectional alignment:

Copyright © 2025 Tony D Barber. All rights reserved.

SRS → Architecture → SADD → Implementation → Testing → Verification
This guarantees:

- No requirement is left unimplemented.
- No architectural component exists without a requirement-driven justification.
- All design choices (RBAC, Redis locks, autoscaling, MongoDB Atlas, SSO, webhooks, audit logs) trace directly back to FRs, NFRs, or ACs from the SRS.

Copyright © 2025 Tony D Barber. All rights reserved.

7. Risks and Mitigation Strategies

7.1 Architectural risks identified and categorized

This system relies on multiple external integrations, distributed components, and performance-critical workflows. The risks listed here focus on architectural weak points, not project-management concerns. Each risk is tied to a specific architectural component or design decision from this SAD and mapped to a realistic mitigation approach.

Risk ID	Category	Risk Description	Impact on System	Mitigation Strategy
AR-01	Integration	**CAS SSO latency or outage** causes login failures or degraded UX.	Users unable to authenticate → registration drop-offs; violates availability NFR.	Add token caching at API Gateway, implement graceful fallback screens, monitor CAS health endpoint, add retry/backoff.
AR-02	Performance	**High registration spikes** overload seat-locking flow (Redis).	Oversubscription, inconsistent waits, slow responses.	Use atomic Redis locks, set Lua scripts for atomic ops, horizontal scaling for FastAPI pods, rate-limiting at API Gateway.
AR-03	Data Consistency	**Race conditions** during simultaneous registrations.	Incorrect seat counts; registration inconsistencies.	Strict use of Redis atomic increments, idempotency keys, and write-through patterns to MongoDB.
AR-04	Security	**Tampering with approval or registration APIs** without correct role permissions.	Unauthorized event edits; FERPA and policy violations.	Enforce RBAC at the Gateway, internal policy checks per micro-service, audit logs on sensitive endpoints.

Copyright © 2025 Tony D Barber. All rights reserved.

Risk ID	Category	Risk Description	Impact on System	Mitigation Strategy
AR-05	Availability	**MongoDB performance degradation** under heavy read/write load.	Slower event search, registration delays, queue buildup.	Use indexed queries, read/write separation via replica sets, connection pooling, and periodic performance monitoring.
AR-06	External Dependency	**Payment gateway webhook failure** or delayed delivery.	Users complete payments but registration isn't confirmed → trust issues.	Implement retries with backoff, idempotent webhook handlers, queue processing, audit event logs.
AR-07	Reliability	**Notification service delay/failure** (email provider).	Missed reminders, delayed confirmations, poor UX.	Async workers with retry policies, fallback SMTP pathway, queue monitoring, and daily alerting.
AR-08	Deployment	**Container misconfiguration or pod crash loops** in Akamai cloud.	Outages or slow endpoints.	Health checks, autoscaling rules, resource quotas, blue-green deploy strategy during updates.
AR-09	Security	**Improper handling of payment data** → PCI-DSS non-compliance.	High-severity security violation; legal repercussions.	Do not store card data, rely fully on approved provider, enforce TLS everywhere, sanitize logs.
AR-10	Observability	**Lack of unified logs/metrics** across services makes debugging slow.	Hard to trace failures, long MTTR.	Centralize logs (Akamai logging or ELK), set p95/p99 alerts, track queue depth, API errors, DB

Copyright © 2025 Tony D Barber. All rights reserved.

Risk ID	Category	Risk Description	Impact on System	Mitigation Strategy
				latency.
AR-11	Scalability	**Event catalog search becomes slow** with large dataset.	Violates SRS NFR (2s response).	Redis caching for hot queries, pagination, index tuning, CDN caching for static assets.
AR-12	Communication	**API Gateway misrouting or rate-limit misconfigurations**.	Random failures or false throttling.	Validate routing rules, stage-based configs (dev/test/prod), smoke tests after deployment.

Table 26: Risk and Mitigation Strategy

7.2 Architectural Justification (Mitigation Approaches)

This risk set maps directly to the architecture decisions made earlier:

- Because the system uses **CAS**, **MongoDB**, **Redis**, and **Payment Gateway**, integration risks are expected.
- Because registration is **performance-critical**, concurrency risks must be explicitly addressed.
- Because waitlists rely on **background async flows**, reliability risks in queue consumption must be covered.
- Because the platform handles registration and attendance data, **FERPA**, **PCI-DSS**, and **NFR-driven security concerns** must be included.

This alignment ensures the mitigation plan reflects both the structure of the system and the operational environment required by the University of Maryland.

Risk-to-Component Matrix (EXTRA CREDIT SECTION)

Component	Related Risks	Why This Component Is Exposed
API Gateway	AR-01, AR-04, AR-12	External-facing boundary; handles CAS, RBAC, rate-limiting; single routing point.

Copyright © 2025 Tony D Barber. All rights reserved.

Auth Service (CAS)	AR-01, AR-04	External dependency; SSO latency; trust boundary for identity and roles.
Event Service	AR-02, AR-11	High-traffic area; event lookups; heavy read patterns; dependent on MongoDB.
Registration Service	AR-02, AR-03, AR-06	High concurrency; seat locking; payment confirmation alignment.
Payment Service	AR-06, AR-09	Relies on external gateway; uses secure callbacks; must meet PCI-DSS.
Notification Service	AR-07	Depends on external email provider; async operations may delay.
MongoDB	AR-03, AR-05, AR-11	Core data store; intensive reads/writes; potential performance bottlenecks.
Redis Cache/Locks	AR-02, AR-03, AR-11	Hot path for registration; concurrency control and caching layer.
Async Queue	AR-06, AR-07	Payment reconciliation and notifications depend on message delivery.
Akamai Cloud Deployment (Pods/Containers)	AR-08, AR-12	Hosting environment; prone to misconfigurations, autoscaling issues, outages.
Log/Monitoring Stack	AR-10	Needed for observability, debugging, and runtime operations.

Copyright © 2025 Tony D Barber. All rights reserved.

Table 27: Risk to Component Matrix

Copyright © 2025 Tony D Barber. All rights reserved.

Appendix A

References

- Grading Rubric provided by the professor
- Software Architecture slides provided by the professor
- Software Architecture Template provided by the professor
- Jira by Atlassian
- Team 4 SRS Team4- Software Requirements Specification (SRS)
- Team 4 PMP Team4-Project Management Plan(PMP)
- Cover Image got it from here
- Layered Architecture image got from here
- Mvc Pattern Image taken from here
- Layered Architecture Image Taken from here
- RBAC Image taken from here
- CI/CD Image taken from here
- Webhooks Image taken from here

Copyright © 2025 Tony D Barber. All rights reserved.

Appendix B

Acronyms

- **PCI DSS -** Payment Card Industry Data Security Standard. It is a set of security standards designed to ensure that all companies that accept, process, store, or transmit credit card information maintain a secure environment.
- **TLS -** Transport Layer Security. It is a cryptographic protocol that ensures secure communication over a computer network by encrypting data transmitted between clients and servers, protecting it from eavesdropping or tampering.
- **WCAG -** Web Content Accessibility Guidelines. It is a set of international standards developed by the W3C (World Wide Web Consortium) that provide recommendations for making web content more accessible to people with disabilities, including those with visual, auditory, physical, or cognitive impairments.
- **REST -** Representational State Transfer
- **API -** Application Programming Interface
- **UMD CAS (Central Authentication Service) -** A single sign-on protocol used by the University of Maryland to authenticate students, faculty, and staff. All users must have a UMD Directory ID to access the system.
- **SSO -** Single Sign On
- **JWT -** JSON Web Tokens
-

Copyright © 2025 Tony D Barber. All rights reserved.

Appendix C

Definitions

- **asyncio** is Python's built-in asynchronous programming framework that lets your program handle many tasks at once without blocking.
- **uvicorn** is a high-performance web server that runs asynchronous Python apps built with frameworks like FastAPI.
- **Horizontal Scaling -** Horizontal scaling is the process of increasing system capacity by adding more machines or instances to distribute the workload across multiple servers.
- **User Engagement Processes -** User Engagement Processes refers to the key actions users perform on the platform, including below:
    - Event browsing
    - Registration
    - Submitting feedback.
- **System Components -** Below are the system components that the application has:
    - Frontend
    - Backend
    - Database

Copyright © 2025 Tony D Barber. All rights reserved.